

Fondamenti di Reti di Calcolatori

NECKER

27 Agosto 2025

Indice

1 Reti e Trasmissione Dati	6
1.1 Bus e Buffer	6
1.2 Porte I/O e Clock	6
1.3 Calcolo del Tempo di Trasmissione	6
2 Teorema di Nyquist-Shannon	6
3 Protocollo TCP	6
3.1 Gestione della Congestione	6
3.2 Problematiche nelle Reti Wireless	6
4 ACK in TCP	6
4.1 ACK Cumulativi	6
4.2 ACK Duplicati	7
4.3 Gestione dei Duplicati	7
5 Il Modello OSI	7
5.1 I sette livelli del modello OSI	7
5.2 Suddivisione Funzionale	7
5.3 impacchettamento	7
5.4 esempio dettagliato	7
6 Architettura e Funzionamento dei Livelli	9
6.1 Il Livello Data-Link (Livello 2)	9
6.1.1 Comunicazione con i livelli adiacenti	9
6.1.2 Framing e Delimitazione	9
6.2 Bit Stuffing	9
6.2.1 Funzionamento	9
6.2.2 Punto di chiarezza	9
7 Affidabilità e Utilizzo del Canale	10
7.1 Sequenziamento dei Frame e degli ACK	10
7.2 Efficienza della Trasmissione e Utilizzo del Canale	10
7.3 Efficienza della Trasmissione e Utilizzo del Canale	10
7.4 Analisi degli Esempi Pratici	10
8 Protocolli a Finestra: Go-back-N e Selective Repeat	11
8.1 Go-back-N	11
8.2 Selective Repeat	12
8.3 Confronto: NAK vs. ACK Duplicati	12
8.4 Meccanismi di Conferma e Ritrasmissione a Livelli 2 e 4	12
9 Gestione della Ritrasmissione con Numerazione Ciclica	12
9.1 Protocollo Go-back-N e la regola $k < 2^m - 1$	13
9.2 Protocollo Selective Repeat e la regola $k \leq 2^{m-1}$	13

10 Topologie e Accesso al Canale nelle Reti Locali (LAN)	14
10.1 Topologie di Rete LAN: Broadcast vs. Point-to-Point	14
10.2 Problemi e Soluzioni nella Trasmissione su Reti Broadcast	14
10.2.1 Accesso Deterministico: Token Ring	14
10.3 Schemi di Accesso CSMA e Varianti Persistenti	14
10.3.1 Accesso Non Deterministico: CSMA/CD in Ethernet	15
10.4 Il meccanismo di Binary Exponential Backoff (BEB)	15
11 Codifica Manchester	16
12 Efficienza e Topologie di Rete Ethernet	16
12.1 Efficienza del canale in una rete Ethernet	16
12.2 L'evoluzione delle topologie: da Hub a Switch	16
12.2.1 Il concetto di Dominio di Collisione	17
12.2.2 Soluzione: l'introduzione del Bridge	17
12.2.3 Ottimizzazione della rete: l'introduzione dello Switch	17
12.3 Configurazioni ibride	17
13 Ottimizzazione in Reti Ethernet ad Alta Velocità	18
13.1 Problema della Latenza e Dimensione del Frame	18
13.1.1 Calcolo del Tempo di Propagazione	18
13.2 Il Ruolo del Livello Data Link (Livello 2)	18
14 Virtual LAN (VLAN)	18
14.1 Vantaggi delle VLAN	18
14.2 Funzionamento e Formato dei Frame	19
14.3 Comunicazione tra VLAN	19
15 Struttura di un Pacchetto IPv4	20
15.1 Campi dell'Header IPv4	20
15.2 Gestione della Frammentazione	20
15.3 Esempio di Frammentazione	20
15.3.1 Passaggio su Rete Token Ring (MTU = 4000 B)	21
15.3.2 Passaggio su Rete Ethernet (MTU = 1500 B)	21
16 Tecniche di Indirizzamento IP	21
16.1 Classi di Indirizzi IP	21
16.1.1 Esempi di Suddivisione	22
16.2 Subnetting	22
16.2.1 Esempio Pratico	22
17 CIDR (Classless Inter-Domain Routing)	23
17.1 Struttura e Assegnazione CIDR	23
17.2 Esempi di Allocazione e Calcolo CIDR	23
17.2.1 Esempio 1: Calcolo della Maschera CIDR	23
17.2.2 Esempio 2: Primo e Ultimo Indirizzo Disponibile	23
17.2.3 Esempio 3: Appartenenza a una Rete	23
18 NAT (Network Address Translation)	23
18.1 Cos'è il NAT e a cosa serve?	23
18.2 Struttura del NAT	24
18.3 Funzionamento del NAT	24
18.4 Problema della gestione delle risposte	24
18.5 Violazione del modello OSI: il NAT e la manipolazione dei livelli	24
18.6 Lavoro del NAT su ogni pacchetto	25
18.7 NAT e server accessibili dall'esterno	25

19 ARP (Address Resolution Protocol) (come trovare il mac di un dispositivo se non lo conosco), (arp si fa solo nelle reti locali dato che il MAC serve per trasmettere l'informazione nel canale fisico ip nel logico)	25
19.1 Come risolve un IP in un MAC address?	25
19.2 Funzionamento dettagliato di ARP	25
19.3 Formato del pacchetto ARP	26
19.4 ARP su reti diverse: il ruolo del router (Gateway)	26
19.4.1 Come si risolve il problema?	26
19.4.2 Proxy ARP	27
20 DHCP, ICMP e Routing	28
20.1 Rinnovo e Rilascio dell'IP	28
20.2 Formato ICMP	28
20.3 Routing vs. Forwarding vs. ARP	28
21 Protocolli di Routing a Vettore di Distanza	28
21.1 Principi di Funzionamento	28
21.2 RIP (Routing Information Protocol)	28
21.3 Problemi dei Protocolli a Vettore di Distanza	28
21.3.1 Bouncing Effect	29
21.3.2 Count to Infinity	29
21.4 Soluzioni e Correzioni	29
21.4.1 Limiti delle soluzioni: perché Triggered Updates e Split Horizon possono fallire	29
22 Limiti e Applicazioni	29
22.1 Inapplicabilità su Larga Scala	29
23 Protocolli di Routing Link State	30
23.1 Principi di Funzionamento (Link State)	30
23.2 Vantaggi e Svantaggi dei Protocolli Link State	30
23.3 OSPF (Open Shortest Path First)	30
23.3.1 Struttura di una Rete OSPF: Le Aree	30
23.3.2 Ottimizzazioni in OSPF: Designated Router (DR)	31
24 Routing su Larga Scala e Architetture Moderne	31
24.1 Routing tra Sistemi Autonomi: BGP	31
24.2 Evoluzione: Software Defined Networking (SDN)	31
24.3 Tecniche di Trasporto: Tunneling	31
25 Border Gateway Protocol (BGP)	32
25.1 Path Vector Routing e gestione del Bouncing Effect	32
26 Problematiche del routing tradizionale	32
27 MPLS: Multi-Protocol Label Switching	32
28 Funzionamento del MPLS	32
29 Gestione delle etichette	33
30 Costruzione delle tabelle di switching	33
31 Formato del pacchetto MPLS	33
32 Spanning tree	34
33 Quality Of Service (QoS) nelle reti di comunicazione	35
33.1 Gestione delle code: Code di Ingresso (Input Queuing)	35
33.1.1 Random Early Detection (RED)	35
33.1.2 ICMP Choke Packets	35
33.2 Gestione delle code: Code di Uscita (Output Queuing)	35
33.2.1 Il Problema della Starvation	35
33.2.2 Weighted Fair Queuing (WFQ) come soluzione	36

33.2.3	Weighted RED (WRED)	36
33.3	Traffic Shaping	36
33.3.1	Token Bucket	36
33.3.2	Call Admission Control (CAC)	36
33.4	struttura protocolli router	36
34	IPv6: Caratteristiche e Transizione da IPv4	37
34.1	Caratteristiche Principali e Differenze con IPv4	37
34.1.1	Formato e Struttura Gerarchica degli Indirizzi	37
34.2	Meccanismi di Compatibilità e Transizione	37
34.2.1	Dual Stack	37
34.2.2	Tunneling	37
34.2.3	Traduzione (NAT64)	37
35	Protocollo TCP (Transmission Control Protocol)	38
35.1	Gestione delle Connessioni Multiple su un Server	38
35.2	Processo di Connessione: Primitiva Socket	38
35.2.1	Lato Client	38
35.2.2	Lato Server	38
35.2.3	Scambio Dati	38
35.3	Header TCP	38
35.3.1	Pseudo-Header e Checksum	39
35.4	Fase di Apertura: Three-Way Handshake	39
35.5	Fase di Trasferimento Dati e Controllo di Flusso	39
35.5.1	Gestione dei Blocchi: Persist Timer	39
35.6	Gestione delle Connessioni Inattive: Keep-Alive Timer	39
36	Controllo di Congestione in TCP	40
36.1	Meccanismi di Ritrasmissione: RTO vs Fast Retransmit	40
36.2	Ottimizzazione del Flusso: Algoritmi di Nagle e Clark	40
36.2.1	per evitare di intasare la banda per ogni singolo carattere si usano questi 2 metodi:	40
36.3	Controllo della Congestione (Congestion Window - cwnd)	40
36.3.1	Fasi di Crescita della Finestra	40
36.3.2	Reazione alla Perdita di Pacchetti (Congestione)	41
36.3.3	self-clocking VS cwnd	41
37	Meccanismi Avanzati in TCP	42
37.1	Calcolo Dinamico del Timeout (RTO)	42
37.2	Regola di Karn: Gestire l'Ambiguità nelle Ritrasmissioni	42
37.3	Migliorare l'Affidabilità: Timestamp e SACK	42
37.4	Chiusura della Connessione (Four-Way Handshake)	42
38	UDP (User Datagram Protocol)	44
38.1	Caratteristiche Principali	44
38.2	Quando Usare UDP?	44
39	Protocolli Real-Time: RTP e RTCP	45
39.1	Caratteristiche Generali	45
39.2	RTP (Real-time Transport Protocol) - Il Flusso Dati	45
39.3	RTCP (RTP Control Protocol) - Il Flusso di Controllo	45
40	DNS (Domain Name System)	46
40.1	Struttura Gerarchica e Risoluzione dei Nomi	46
40.2	Record DNS	46
41	Architettura della Posta Elettronica (Email)	47
41.1	Componenti Chiave	47
41.2	Flusso di Invio e Ricezione di un'Email	47
41.3	Protocolli per l'Accesso alla Posta	47
41.4	Gestione degli Allegati: MIME e Base64	47
41.5	struttura SMTP	47

42 FTP (File Transfer Protocol)	48
42.1 Architettura a Due Connessioni	48
42.2 Modalità di Connessione Dati: Attiva vs Passiva	48
42.3 Separazione tra Piano di Controllo e Piano Dati	48
43 Fasi di Assegnazione IP in DHCP	49
44 HTTP (HyperText Transfer Protocol)	50
44.1 HTTP/1.0: Connessioni Non Permanenti	50
44.2 HTTP/1.1: Connessioni Permanenti e Pipelining	50
44.3 HTTP/2: Multiplexing su TCP	50
44.4 HTTP/3: Il Passaggio a QUIC su UDP	50
45 Differenza tra HTTP e HTTPS	51

1 Reti e Trasmissione Dati

1.1 Bus e Buffer

I dati si muovono all'interno del computer in **parallelo** attraverso i **bus**, canali di comunicazione che collegano le varie componenti. I bit vengono temporaneamente immagazzinati in un'area di memoria chiamata **buffer** prima di essere inviati.

1.2 Porte I/O e Clock

La **porta di I/O** converte i dati dal formato parallelo a quello **seriale** (un bit alla volta) per la trasmissione sui canali di rete. Questa porta è **bidirezionale**, permettendo lo scambio di dati in entrambe le direzioni. Il ritmo di trasmissione è scandito da un **clock**, un segnale di sincronizzazione che determina la velocità (bps) con cui i bit vengono inviati.

1.3 Calcolo del Tempo di Trasmissione

Il tempo necessario per trasmettere un blocco di bit si calcola dividendo il numero di bit per la velocità di trasmissione.

$$T_x = \frac{\text{Numero di bit}}{\text{Velocità di trasmissione}}$$

Esempio: Per trasmettere 1000 bit a 1 Mbps (1 milione di bit al secondo):

$$T_x = \frac{1000}{10^6} = 10^{-3} \text{ s} = 1 \text{ ms}$$

2 Teorema di Nyquist-Shannon

Questo teorema stabilisce i requisiti per digitalizzare un segnale analogico senza perdere informazioni. Il principio fondamentale è che la **frequenza di campionamento** (f_s) deve essere almeno il doppio della **massima frequenza** (f_{max}) presente nel segnale originale.

$$f_s \geq 2 \cdot f_{max}$$

Esempio: La voce umana ha una f_{max} di circa 4000 Hz. Di conseguenza, la frequenza di campionamento minima necessaria è 8000 Hz (8 kHz).

3 Protocollo TCP

invia blocchi di pacchetti e non singoli pacchetti

3.1 Gestione della Congestione

TCP è progettato per garantire una trasmissione affidabile e gestire la **congestione** della rete (i router hanno dei buffer, se sono pieni i pacchetti che arrivano vengono scartati). La sua strategia consiste nel rallentare l'invio dei dati quando rileva la perdita di pacchetti, presumendo che ciò sia dovuto a un eccessivo traffico (poiché il TCP fu inventato quando esistevano solo le reti cablate, la perdita di un pacchetto, essendo rara, spesso indicava l'overflow dei buffer e in automatico un rallentamento del flusso di dati inviato dal mittente).

3.2 Problematiche nelle Reti Wireless

Le reti wireless sono soggette a errori sul canale (interferenze, ostacoli). TCP può interpretare erroneamente la perdita di pacchetti come segnale di congestione, riducendo la velocità di trasmissione in modo inutile e peggiorando le prestazioni della rete.

4 ACK in TCP

4.1 ACK Cumulativi

Gli **ACK cumulativi** sono usati per confermare la ricezione di tutti i pacchetti in ordine, fino a un certo punto della sequenza. ad esempio se il destinatario riceve: 1,2,3,5,6 invia ack per: 1,2,3 se poi dopo il 6 riceve il 4 allora invia ack per il 6 (quindi il 4,5 no poiché li ha tutti fino al 6)

4.2 ACK Duplicati

Gli **ACK duplicati** sono usati per segnalare la perdita di un pacchetto. Vengono inviati quando un pacchetto arriva fuori sequenza, e al ricevimento di un certo numero di questi, il mittente ritrasmette immediatamente il pacchetto mancante. ad esempio se si riceve 1,2,3,5,6 per 5 e 6 invio un ack duplicato per il 4

4.3 Gestione dei Duplicati

Se, dopo una ritrasmissione, il ricevente riceve un pacchetto identico, lo **scarta silenziosamente**.

5 Il Modello OSI

- Il modello OSI (Open Systems Interconnection) è una suddivisione concettuale delle funzioni di rete in sette livelli. L'obiettivo è separare la complessità della comunicazione di rete in parti più semplici, facilitando la standardizzazione e la risoluzione dei problemi.

5.1 I sette livelli del modello OSI

1. **Livello 1 - Fisico:** Si occupa della trasmissione dei bit a livello hardware (cavi, onde radio, ecc.). Gestisce il segnale fisico.
2. **Livello 2 - Data Link (Collegamento dati):** Gestisce il trasferimento affidabile dei dati tra due nodi **adiacenti** su una stessa rete locale (LAN). (non solo punto a punto)
3. **Livello 3 - Network (Rete):** Si occupa dell'instradamento (*routing*) dei pacchetti tra reti diverse. A questo livello opera l'**IP** (Internet Protocol) e il router.
4. **Livello 4 - Transport (Trasporto):** Garantisce un trasferimento dati affidabile tra host finali, gestendo la segmentazione e il riassemblaggio dei dati. I protocolli chiave sono **TCP** e **UDP**.
5. **Livello 5 - Session (Sessione):** Gestisce l'apertura, il mantenimento e la chiusura delle **sessioni di comunicazione** tra le applicazioni.
6. **Livello 6 - Presentation (Presentazione):** Gestisce la formattazione dei dati, la compressione e la crittografia, assicurando che le informazioni siano leggibili da sistemi diversi.
7. **Livello 7 - Application (Applicazione):** Rappresenta il livello più vicino all'utente. Include i protocolli che le applicazioni usano per accedere ai servizi di rete (es. HTTP, FTP, SMTP).

5.2 Suddivisione Funzionale

I sette livelli possono essere raggruppati in due macro-categorie:

- **Livelli Inferiori (1-4):** Si occupano del **trasporto dei dati**. Gestiscono i meccanismi fisici e logici per spostare i dati da un punto all'altro della rete.
- **Livelli Superiori (5-7):** Si occupano dell'**elaborazione dei dati per le applicazioni**. Lavorano a stretto contatto con il software per garantire che i dati siano formattati e gestiti correttamente per l'utente finale.

5.3 impacchettamento

quanto un pacchetto viene inviato, esso passa per tutti i livelli e ciascuno lo incapsula in una sorta di scatola (aggiungendoci un header), arrivato a destinazione il pacchetto viene spaccettato con il processo opposto

5.4 esempio dettagliato

Supponiamo che tu voglia inviare il messaggio "Ciao" dal tuo computer (IP: 192.168.1.1, MAC: A) a un server web (IP: 203.0.113.10, MAC: B).

Livello 7 - Applicazione: L'applicazione (es. un browser web) crea i dati: "Ciao".

Livello 4 - Trasporto: Aggiunge l'header TCP con i numeri di porta (es. 80 per il server). Il dato diventa: [TCP Header]Ciao.

Livello 3 - Rete: Aggiunge l'header IP con gli indirizzi IP di origine e destinazione. Il dato diventa: [IP Header (192.168.1.1 → 203.0.113.10)] [TCP Header] Ciao.

Livello 2 - Data Link: Aggiunge l'header MAC e il trailer, specificando gli indirizzi MAC di origine e destinazione (che sono quelli del prossimo router). Il dato diventa: [MAC Header (A → Router)] [IP Header] ... [Trailer].

Livello 1 - Fisico: Il frame viene convertito in segnali elettrici e trasmesso.

Quando il pacchetto arriva al router, viene **deincapsulato** fino al Livello 3, dove il router legge l'indirizzo IP di destinazione e decide il prossimo percorso. A quel punto, il pacchetto viene **incapsulato di nuovo** con un nuovo header MAC per il prossimo router sul percorso. Questo processo si ripete fino a quando il pacchetto raggiunge il server finale, dove viene completamente deincapsulato fino al Livello 7.

6 Architettura e Funzionamento dei Livelli

6.1 Il Livello Data-Link (Livello 2)

Il Livello Data-Link si occupa del trasferimento affidabile dei dati tra due nodi **adiacenti** su una stessa rete locale. Il suo campo d'azione è limitato e non ha una visione globale della rete. (esiste un livello data-link per ogni nodo del router, in pratica per ogni collegamento possibile).

6.1.1 Comunicazione con i livelli adiacenti

Il Livello 3 (Network) passa i pacchetti al Livello 2, che può gestire la trasmissione in due modalità:

- **Affidabile:** invia i dati e attende una conferma di ricezione. (una specie di TCP)
- **Non affidabile (Best Effort):** invia i dati senza garanzia di ricezione. Questo approccio è spesso preferito per non rallentare la rete con continue conferme su ogni singolo "salto" tra nodi. (una sorta di UDP) usato nelle porte ethernet.

6.1.2 Framing e Delimitazione

Per distinguere i frame di dati dal canale in stato di inattività (idle, una sequenza continua di '1'), il Livello Data-Link utilizza il **framing**. Aggiunge una speciale sequenza di bit, chiamata **flag**, all'inizio e alla fine di ogni frame.

- Il flag è la sequenza fissa di bit 01111110.
- L'unico scopo del flag è delimitare l'inizio e la fine della trasmissione dati.

ATTENZIONE: un frame non è un blocco di pacchetti

6.2 Bit Stuffing

Il bit stuffing è una tecnica utilizzata dal Livello Data-Link per evitare che la sequenza di flag (01111110) compaia accidentalmente all'interno dei dati utili, dove potrebbe essere interpretata erroneamente come un delimitatore.

6.2.1 Funzionamento

1. **Lato Trasmettitore:** Il trasmettitore scansiona i dati da inviare. Se incontra una sequenza di **cinque '1' consecutivi**, inserisce automaticamente un bit '0' subito dopo il quinto '1'.
2. **Lato Ricevitore:** Il ricevitore esegue il processo inverso. Quando riceve una sequenza di **cinque '1' consecutivi**, controlla il bit successivo.
 - Se il bit successivo è '0', lo considera un bit di stuffing e lo **scarta**.
 - Se il bit successivo è '1', riconosce la sequenza come un flag (01111110) e conclude la ricezione del frame.

6.2.2 Punto di chiarezza

Il bit di stuffing viene rimosso dal ricevitore senza che i dati originali vengano alterati. La logica è basata sulla certezza che dopo cinque '1' consecutivi, qualsiasi '0' sia un bit di stuffing aggiunto dal mittente, a meno che non si tratti del flag di fine frame.

7 Affidabilità e Utilizzo del Canale

7.1 Sequenziamento dei Frame e degli ACK

L'affidabilità di un protocollo di comunicazione richiede meccanismi che prevengano la perdita o la duplicazione dei dati. La soluzione a questo problema è l'utilizzo di **numeri di sequenza** sia per i frame che per i relativi ACK.

- **Numerazione dei Frame:** Ogni frame inviato dal mittente è identificato da un numero di sequenza progressivo. Il ricevitore si aspetta di ricevere i frame in un ordine specifico.
- **Ruolo dei Numeri di Sequenza degli ACK:** È cruciale che anche gli ACK (le conferme di ricezione) contengano un numero di sequenza che si riferisca al frame ricevuto. Questo serve a:
 - **Evitare Ambiguità:** Senza un numero di sequenza, un ACK potrebbe essere interpretato come una conferma per un frame precedente, specialmente se l'ACK di un frame precedente è arrivato in ritardo.
 - **Gestire la Ritrasmissione:** Se il mittente invia il Frame 0 ma il suo ACK si perde, il mittente ritrasmette il Frame 0. Quando il ricevitore lo riceve, lo riconosce come duplicato grazie al numero di sequenza e lo scarta. Se l'ACK non avesse un numero, il mittente non avrebbe modo di sapere se una conferma arrivata in ritardo si riferisce a una trasmissione precedente o meno.

7.2 Efficienza della Trasmissione e Utilizzo del Canale

Un protocollo non deve essere solo affidabile, ma anche efficiente. L'efficienza si misura con l'**utilizzo del canale** (U), che indica quanto la capacità del canale viene sfruttata per trasmettere dati utili.

$$U = \frac{t_x}{T}$$

- t_x (**Tempo di trasmissione**): Il tempo necessario per inviare un frame sul canale.
- T (**Tempo totale**): Il tempo che intercorre tra l'inizio della trasmissione del frame e la ricezione del suo ACK.

$$T = t_x + 2t_p$$

Nei protocolli di base, come il **Stop-and-Wait**, il trasmettitore invia un solo frame alla volta e rimane inattivo in attesa dell'ACK. Questo metodo è estremamente **inefficiente** perché gran parte del tempo il canale rimane vuoto.

L'efficienza è influenzata principalmente da:

- **Tempo di Trasmissione** (t_x): Dipende dalla dimensione del frame e dalla banda del canale.
- **Tempo di Propagazione** (t_p): Il tempo impiegato dal segnale per percorrere la distanza tra mittente e ricevitore.

7.3 Efficienza della Trasmissione e Utilizzo del Canale

L'efficienza di un protocollo si misura con l'**utilizzo del canale** (U), che quantifica quanto la banda disponibile viene usata per trasmettere dati utili. Per un protocollo *Stop-and-Wait*, che invia un solo frame alla volta, la formula è:

$$U = \frac{t_x}{T}$$

Nei protocolli a finestra, come la *Sliding Window* (segue logica TCP), la formula cambia per riflettere il numero di frame (k) che possono essere inviati prima di attendere una conferma.

$$U = k \cdot \frac{t_x}{T}$$

7.4 Analisi degli Esempi Pratici

Esaminiamo come il tempo di propagazione influenzi l'efficienza.

Scenario 1: Canale Corto

- **Larghezza di banda:** 10 Mbps
- **Distanza:** 2 km
- **Dimensione pacchetti:** 1000 bit

Calcoli:

- $t_x = \frac{1000}{10^7} = 0.1 \text{ ms}$
- $t_p = \frac{2 \cdot 10^3}{2 \cdot 10^8} = 0.01 \text{ ms}$ (velocità del segnale su cavo: $2 \cdot 10^8 \text{ m/s}$)
- $T = t_x + 2t_p = 0.1 + 0.02 = 0.12 \text{ ms}$
- **Utilizzo del canale (U):** $U = \frac{0.1}{0.12} \approx 0.83$ (83% di utilizzo)

In questo caso, t_x è molto maggiore di t_p , rendendo il protocollo già abbastanza efficiente anche con il metodo Stop-and-Wait.

Scenario 2: Canale Lungo

- **Larghezza di banda:** 10 Mbps
- **Distanza:** 20 km
- **Dimensione pacchetti:** 1000 bit

Calcoli:

- $t_x = \frac{1000}{10^7} = 0.1 \text{ ms}$
- $t_p = \frac{20 \cdot 10^3}{2 \cdot 10^8} = 0.1 \text{ ms}$ (velocità del segnale su cavo: $2 \cdot 10^8 \text{ m/s}$)
- $T = t_x + 2t_p = 0.1 + 0.2 = 0.3 \text{ ms}$
- **Utilizzo del canale (U):** $U = \frac{0.1}{0.3} \approx 0.33$ (33% di utilizzo)

In questo scenario, il tempo di propagazione è significativo e riduce notevolmente l'efficienza. Il trasmettitore spreca una quantità considerevole di tempo inattivo, in attesa dell'ACK. Per massimizzare l'utilizzo del canale, è necessario inviare più frame in sequenza. Poiché T è 3 volte t_x , il trasmettitore deve inviare almeno **3 frame** per ogni ciclo per mantenere il canale occupato e portare l'utilizzo vicino al 100%.

8 Protocolli a Finestra: Go-back-N e Selective Repeat

I protocolli a finestra, come la *Sliding Window* (seguita da TCP), aumentano l'efficienza del canale consentendo al trasmettitore di inviare più frame consecutivamente prima di ricevere una conferma. A seconda di come gestiscono le perdite, si distinguono due approcci principali.

8.1 Go-back-N

In questo protocollo, il trasmettitore invia una sequenza di frame, ma il ricevitore ha un buffer di dimensione limitata, in genere 1. Questo significa che il ricevitore può accettare solo frame che arrivano in ordine.

- **Gestione degli errori:** Se un frame (es. il frame 3) viene perso o arriva danneggiato, il ricevitore ignora tutti i frame successivi che riceve (es. 4, 5, 6), poiché sono "fuori sequenza".
- **Ritrasmissione:** Il ricevitore può segnalare la perdita inviando un **NAK** (Negative Acknowledgment) per il frame mancante. Il trasmettitore, ricevendo questa notifica o a causa di un timeout, ritrasmette il frame mancante e **tutti i frame successivi** che aveva già inviato. Questa strategia è semplice da implementare ma è **inefficiente**, poiché spreca banda ritrasmettendo frame che il ricevitore aveva già ricevuto correttamente. (il NAK viene inviato una volta sola per non intasare la rete, se viene perso ci penserà il timer del pacchetto a reinviare automaticamente il pacchetto)

ATTENZIONE: (usa cumulativi e NAK ma fanno la stessa cosa)

8.2 Selective Repeat

Questo protocollo è più efficiente. Il ricevitore è dotato di un **buffer di ricezione più grande** che gli permette di accettare e memorizzare i frame che arrivano correttamente, anche se sono fuori sequenza. (ATTENZIONE: NAK è l'equivalente del ACK duplicato per il protocollo TCP)

- **Gestione degli errori:** Quando un frame viene perso (es. frame 3), il ricevitore continua ad accettare e a memorizzare i frame successivi (es. 4, 5, 6).
- **Ritrasmissione:** Il trasmettitore, rilevando un timeout per il frame 3, o su richiesta del ricevitore (es. con un NAK selettivo), ritrasmette **soltanto il frame mancante**.
- **Vantaggi:** Questo approccio riduce drasticamente il numero di ritrasmissioni e migliora l'efficienza del canale. Quando tutti i frame fino a un certo punto sono stati ricevuti, il ricevitore li passa in blocco al livello superiore.

8.3 Confronto: NAK vs. ACK Duplicati

I meccanismi di gestione delle perdite nei protocolli a finestra possono essere gestiti in modo esplicito (NAK) o implicito (ACK duplicati).

- **NAK (Negative Acknowledgment):** È un messaggio di controllo esplicito inviato dal ricevitore per notificare che un pacchetto specifico è andato perso o corrotto. Il suo scopo è avviare una ritrasmissione immediata e mirata. Viene spesso associato ai protocolli Go-back-N.
- **ACK Duplicati:** Sono una tecnica utilizzata da **TCP**. Il protocollo non usa NAK espliciti. Se un pacchetto è perso, il ricevitore continua a inviare ACK che confermano l'ultimo pacchetto **in sequenza** ricevuto correttamente. Ad esempio, se si riceve 1, 2, 4, 5, il ricevitore risponderà con un ACK per il 2 quando riceve il 4 e di nuovo un ACK per il 2 quando riceve il 5. Quando il mittente riceve un numero predefinito di questi ACK duplicati (generalmente 3), presume che il pacchetto successivo (in questo caso, il 3) sia andato perso e lo ritrasmette.

Questo approccio permette a TCP di recuperare le perdite senza attendere il timeout, migliorando le prestazioni.

8.4 Meccanismi di Conferma e Ritrasmissione a Livelli 2 e 4

- **Livello 4 (TCP)**
 - **ACK cumulativo:** conferma tutti i byte ricevuti fino a un certo numero di sequenza.
 - **ACK duplicati:** segnalano al mittente che è stato ricevuto un pacchetto fuori ordine.
 - **SACK (Selective Acknowledgment):** permette di indicare esattamente quali segmenti sono stati ricevuti correttamente.
 - Questi meccanismi garantiscono affidabilità end-to-end e gestione della congestione.
- **Livello 2 (Data Link)**
 - Gli ACK generalmente confermano *frame singolo* o, in protocolli come Go-Back-N/Selective Repeat, il *prossimo frame atteso*.
 - Non cumulativo come TCP, la gestione avanzata dei pacchetti fuori ordine è limitata.
 - **Ethernet:** best effort, nessun ACK, nessun NAK.
 - **Wi-Fi (IEEE 802.11):** ACK obbligatorio per ogni frame; assenza di ACK = ritrasmissione; non usa NAK esplicito.
 - **WAN / PPP / HDLC:** ACK + NAK possibili, permettono ritrasmissioni immediate in caso di errore.

9 Gestione della Ritrasmissione con Numerazione Ciclica

Quando un protocollo a finestra utilizza una numerazione dei frame ciclica, che si resetta dopo aver raggiunto un valore massimo, è cruciale che il ricevitore sia in grado di distinguere un pacchetto ritrasmesso da un pacchetto nuovo con lo stesso numero di sequenza. Questo problema di ambiguità viene risolto in modo diverso a seconda del protocollo.

9.1 Protocollo Go-back-N e la regola $k < 2^m - 1$

Nel protocollo Go-back-N, il ricevitore ha un buffer di dimensione limitata e accetta solo i frame che arrivano in ordine. Se riceve un frame fuori sequenza, lo scarta silenziosamente. Questa logica, apparentemente semplice, è la chiave per evitare ambiguità, a patto che la dimensione della finestra di trasmissione (k) sia inferiore alla dimensione del numero di sequenza.

Per un numero di sequenza di m bit (che produce 2^m valori distinti), la finestra di trasmissione (k) deve soddisfare la condizione:

$$k < 2^m$$

Esempio di ambiguità: Consideriamo un numero di sequenza di 3 (valori da 0 a 2) e una finestra di trasmissione di dimensione $k = 3$. Il mittente può inviare i frame 0, 1, 2.

1. Il mittente invia Frame 0, 1, 2.
2. Il ricevitore riceve Frame 0,1,2 e invia gli ACK relativi. tutti gli ACK si perdono, ora il destinatario si aspetta 0 (il quarto pacchetto)
3. poichè non arrivano gli ACK 0,1,2 il mittente reinvia 0,1,2 (i primi 3 pacchetti)
4. il destinatario li riceve ma li interpreta come pacchetti 4,5,6 e non come quelli prima: ERRORE.

Esempio di ambiguità: Immaginiamo che il mittente invii i pacchetti 0, 1, 2 e poi l'ACK di 0 si perda. Il mittente ritrasmette il pacchetto 0. Il ricevitore, che si aspetta il pacchetto 3, riceve il pacchetto 0 e lo scarta, poiché non è il pacchetto che si aspetta. Questo comportamento previene qualsiasi ambiguità.

ATTENZIONE: se la finestra (n di frame inviati per volta) fosse uguale alla sequenza di trasmissione (numerazione dei frame), si verificherebbe ambiguità poichè ci si aspetta un pacchetto che potrebbe essere uno di quelli persi o uno nuovo

9.2 Protocollo Selective Repeat e la regola $k \leq 2^{m-1}$

A differenza di Go-back-N, in Selective Repeat il ricevitore ha un buffer per memorizzare i pacchetti che arrivano fuori sequenza. Questa caratteristica, se da un lato rende il protocollo più efficiente, dall'altro introduce un potenziale problema di ambiguità molto più grave.

Se la dimensione della finestra di trasmissione (k) non è sufficientemente piccola, un pacchetto ritrasmesso con un certo numero di sequenza potrebbe essere confuso con un nuovo pacchetto. Per evitare ciò, la dimensione della finestra di trasmissione e di ricezione non deve mai superare la metà dello spazio di sequenza disponibile.

Per un numero di sequenza di m bit (valori da 0 a $2^m - 1$), la finestra di trasmissione (k) deve soddisfare la condizione:

$$k \leq \frac{2^m}{2} \iff k \leq 2^{m-1}$$

Esempio di ambiguità (Caso errato): Numero di sequenza di 3 bit (valori da 0 a 7, $2^3 = 8$) e una finestra di trasmissione e ricezione di dimensione $k = 5$.

1. Il mittente invia Frame 0, 1, 2, 3, 4.
2. I Frame 0, 1 si perdono. Il ricevitore riceve e bufferizza i frame 2, 3, 4.
3. Il mittente riceve gli ACK di 0 e 2, 3, 4. Sposta la finestra e invia i frame 5, 6, 7 e 0.
4. Il mittente ritrasmette il Frame 1.
5. Il ricevitore, che ha nel buffer i frame 2, 3, 4 e si aspetta il Frame 1, lo riceve correttamente e lo accetta.

Il problema nasce quando il mittente, a causa di una lunga latenza, ritrasmette il pacchetto 0, che ha un numero di sequenza già utilizzato in precedenza. Il ricevitore potrebbe confonderlo con un nuovo pacchetto, non avendo modo di distinguerlo dal vecchio.

Spiegazione: La regola $k \leq 2^{m-1}$ assicura che un pacchetto ritrasmesso con un numero di sequenza già usato non possa mai apparire nella finestra di ricezione. Quando il numero di sequenza si ripete, il ricevitore ha già "dimenticato" il vecchio pacchetto con lo stesso numero di sequenza, evitando confusione.

10 Topologie e Accesso al Canale nelle Reti Locali (LAN)

10.1 Topologie di Rete LAN: Broadcast vs. Point-to-Point

Le reti locali (LAN) si differenziano dalle topologie a maglia (o point-to-point), in cui ogni nodo è connesso direttamente solo ai suoi adiacenti. Le LAN adottano invece una topologia di tipo **broadcast**, dove ogni nodo è collegato simultaneamente a tutti gli altri. Quando un nodo trasmette un segnale, questo viene ricevuto da tutti i nodi sulla rete.

1. **Topologia a stella con hub passivo:** Una delle prime implementazioni broadcast, che utilizza un hub come centro di una stella. L'hub non ha intelligenza e si limita a ripetere il segnale su tutte le sue porte, propagando il broadcast.
2. **Topologia a bus lineare:** Un'alternativa storica, in cui tutte le stazioni condividono un unico mezzo trasmissivo. Le prime versioni di Ethernet utilizzavano questa topologia, che concettualmente è equivalente a quella a stella con hub passivo.

10.2 Problemi e Soluzioni nella Trasmissione su Reti Broadcast

La natura condivisa delle reti broadcast introduce due problemi principali: la necessità di indirizzare i pacchetti e la gestione della contesa per l'accesso al canale.

- **Indirizzamento mirato:** Ogni pacchetto contiene gli indirizzi del nodo sorgente e del destinatario. In questo modo, i nodi ricevono tutti i pacchetti, ma elaborano solo quelli a loro destinati.
- **Gestione della contesa:** Per evitare collisioni dovute a trasmissioni simultanee, sono stati sviluppati protocolli di accesso al canale.

10.2.1 Accesso Deterministico: Token Ring

Il protocollo Token Ring utilizza un meccanismo deterministico in cui un **token** (un pacchetto speciale) circola lungo un anello. Solo la stazione che detiene il token può trasmettere. Questo approccio garantisce:

- Accesso equo e senza collisioni.

I suoi svantaggi includono lunghi tempi di attesa e la necessità di un'entità centrale per gestire la rete e recuperare il token in caso di perdita. A causa di questi limiti, la topologia Token Ring è stata progressivamente abbandonata.

10.3 Schemi di Accesso CSMA e Varianti Persistenti

- **CSMA senza CD (Carrier Sense Multiple Access), non persistente**
 - Prima di trasmettere, la stazione verifica se il canale è libero.
 - Se libero → trasmissione immediata.
 - Se occupato → attesa fino a che il canale diventa libero. (non controlla il canale per tot tempo (casuale))
 - *Limite:* le collisioni possono ancora verificarsi perché non c'è collision detection.
- **CSMA 0-persistent**
 - Se il canale è libero → trasmissione immediata.
 - Se occupato → attesa di un intervallo casuale di tempo prima di ritentare.
 - Riduce la probabilità di collisioni rispetto a CSMA puro.
- **CSMA 1-persistent**
 - Se il canale è libero → trasmissione immediata.
 - Se occupato → continua a controllare il canale fino a che diventa libero, poi trasmissione immediata.
 - Maggiore probabilità di collisione se più stazioni aspettano nello stesso istante.
- **CSMA p-persistent** (tipico per canali slotted)
 - Se il canale è libero → trasmissione con probabilità p , altrimenti attesa di uno slot successivo.
 - Se il canale è occupato → attesa fino al prossimo slot libero.
 - Compromesso tra collisioni e ritardo di attesa.

10.3.1 Accesso Non Deterministico: CSMA/CD in Ethernet

A differenza del Token Ring, il protocollo CSMA/CD (Carrier Sense Multiple Access with Collision Detection) gestisce l'accesso al canale in modo non deterministico e distribuito, accettando la possibilità di collisioni.

CSMA/CD è un'evoluzione del protocollo **Aloha**, che si basa su tre concetti chiave:

- **Carrier Sense:** Ogni stazione "ascolta" il canale prima di trasmettere per verificare se è libero.
- **Collision Detection:** Durante la trasmissione, ogni stazione monitora il canale per rilevare collisioni, ovvero la sovrapposizione del proprio segnale con quello di un'altra stazione. In caso di collisione, la trasmissione viene interrotta immediatamente per non sprecare banda.
- **Backoff esponenziale:** Dopo una collisione, la stazione non ritrasmette subito. Aspetta un tempo casuale che aumenta esponenzialmente con il numero di collisioni subite.

Questo approccio, pur essendo non deterministico, garantisce un utilizzo più efficiente del canale rispetto ad Aloha.

10.4 Il meccanismo di Binary Exponential Backoff (BEB)

Il BEB è un algoritmo cruciale in CSMA/CD per gestire le ritrasmissioni dopo una collisione. La sua logica è la seguente:

$$\text{ritardo} = \text{random}(0, 2^i - 1) \times T_{\text{slot}}$$

dove:

- i è il numero di collisioni consecutive subite dalla stazione.
- T_{slot} è il tempo minimo di attesa, definito dallo standard IEEE 802.3.

Definizione dello Slot Time (T_{slot}): Lo Slot Time è il tempo massimo di andata e ritorno di un segnale nella rete, garantendo che una collisione possa essere rilevata da tutte le stazioni. Per una rete Ethernet classica con lunghezza massima di 2500 metri e una velocità di 10 Mbps, lo standard IEEE 802.3 lo definisce in **512 bit time**, che corrispondono a 51.2 microsecondi. Questo valore, superiore al tempo di propagazione teorico, include un margine di sicurezza per garantire il corretto funzionamento in ogni condizione.

11 Codifica Manchester

- **Scopo:** La codifica Manchester viene utilizzata per trasmettere dati digitali in modo che il ricevitore possa sincronizzare il proprio clock con quello del trasmettitore. Ogni bit trasmesso presenta una transizione di tensione al centro del periodo del bit, garantendo il clock recovery.
- **Rappresentazione dei bit:**
 - 0 → transizione *alto* → *basso* a metà bit.
 - 1 → transizione *basso* → *alto* a metà bit.
- **Preambolo e flag iniziali/finali:**
 - All'inizio di una trasmissione Ethernet viene inviato un **preambolo** costituito da 7 byte alternati 10101010 (in Manchester: transizioni regolari), seguito da un **Start Frame Delimiter (SFD)** di 1 byte 10101011.
 - Il preambolo permette al ricevitore di sincronizzare il clock con il trasmettitore.
 - I flag iniziali e finali sono eliminati durante la codifica Manchester perché il ricevitore non ha bisogno di segnali esterni: il clock è ricavato direttamente dalle transizioni dei bit.
- **Gestione di sequenze consecutive di 1 o 0:**
 - Se si trasmettono bit identici consecutivi, ad esempio 1111, la codifica Manchester genera transizioni a metà bit per ogni bit.
 - La transizione iniziale di un nuovo bit identico può essere ignorata dal ricevitore come "preparazione" e non come transizione valida per il clock. Questo evita che la continuità del livello alto o basso falsi la sincronizzazione.
 - In pratica, il ricevitore misura la differenza temporale tra transizioni per determinare correttamente ogni bit, ignorando le transizioni preparatorie quando necessarie.
- **Vantaggio principale:** consente al ricevitore di recuperare il clock direttamente dai dati trasmessi, eliminando la necessità di un canale separato per la sincronizzazione.

Questa tecnica raddoppia la velocità del clock di trasmissione per garantire che le transizioni avvengano esattamente a metà di ogni periodo di bit, permettendo al ricevitore di sincronizzarsi con il flusso di dati.

12 Efficienza e Topologie di Rete Ethernet

12.1 Efficienza del canale in una rete Ethernet

L'efficienza di un canale Ethernet dipende dalla gestione delle **contese**, ovvero la probabilità che più stazioni cerchino di trasmettere contemporaneamente, causando **collisioni**. L'efficienza dell'utilizzo del canale (U) in una rete punto-punto può essere espressa come:

$$U = \frac{t_x}{t_x + 2t_p \times e}$$

dove:

- t_x è il tempo di trasmissione del frame (L_{frame}/B).
- t_p è il tempo di propagazione del segnale (L_{canale}/v_p).
- e è un parametro che tiene conto dell'impatto delle collisioni.

Sostituendo i valori, la formula diventa:

$$U = \frac{1}{1 + 2 \times \frac{B \times L_{\text{canale}}}{L_{\text{frame}} \times v_p} \times e}$$

Questa formula evidenzia che un aumento della banda (B) o della lunghezza del canale (L_{canale}) riduce l'efficienza. Questo accade perché il tempo di propagazione ha un impatto maggiore rispetto al tempo di trasmissione, aumentando il rischio e la durata delle collisioni.

12.2 L'evoluzione delle topologie: da Hub a Switch

Le prime reti Ethernet adottavano topologie broadcast (come il bus o la stella con hub) in cui tutte le stazioni condividevano lo stesso mezzo trasmissivo. L'intera rete costituiva un unico, grande **dominio di collisione**. All'aumentare delle stazioni, la probabilità di collisione cresceva, degradando le prestazioni.

12.2.1 Il concetto di Dominio di Collisione

Un dominio di collisione è un segmento di rete in cui una collisione tra due pacchetti può essere rilevata da tutte le stazioni. Nelle reti con hub, una collisione in qualsiasi punto del cavo si propaga a tutti i nodi, obbligando tutti a fermare la trasmissione e a riavviare il processo di backoff esponenziale.

12.2.2 Soluzione: l'introduzione del Bridge

Per risolvere questo problema, è stato introdotto il **bridge**, un dispositivo intelligente di Livello 2 (Data Link Layer). Il bridge agisce come un "ponte" tra due o più segmenti di rete, separando il traffico tra di essi e, di conseguenza, dividendo un grande dominio di collisione in più domini più piccoli.

- **Funzionamento:** A differenza di un hub, che ripete ciecamente i segnali, il bridge esamina l'indirizzo MAC di destinazione di ogni frame.
- **Filtraggio e inoltra:** Se il mittente e il destinatario si trovano sullo stesso segmento di rete, il bridge scarta il frame. Se il destinatario si trova in un altro segmento, il bridge inoltra il frame solo sulla porta corretta.
- **Tabella di forwarding:** Il bridge costruisce una tabella dinamicamente, associando gli indirizzi MAC delle stazioni alle porte di connessione. Se un indirizzo è sconosciuto, il bridge esegue un *flooding* (inoltra il pacchetto su tutte le porte), imparando la posizione del destinatario non appena questo risponde.

12.2.3 Ottimizzazione della rete: l'introduzione dello Switch

Lo **switch** rappresenta l'evoluzione del bridge, un dispositivo di Livello 2 che ottimizza ulteriormente il traffico di rete e quasi elimina le collisioni.

- **Connessioni punto-a-punto:** A differenza del bridge che collega interi segmenti, lo switch crea un dominio di collisione separato per ogni singola porta. Questo significa che ogni dispositivo collegato allo switch ha un canale dedicato.
- **Eliminazione delle collisioni:** Grazie a queste connessioni dedicate, non avvengono più collisioni tra dispositivi collegati a porte diverse dello switch. Questo rende superfluo l'uso del protocollo CSMA/CD sulla maggior parte della rete, migliorando drasticamente le prestazioni e il throughput.
- **Prestazioni superiori:** Più dispositivi possono comunicare contemporaneamente senza interferenze, riducendo la latenza e aumentando l'efficienza complessiva.

12.3 Configurazioni ibride

Nelle reti moderne, la configurazione più efficiente prevede l'uso di **switch** come dispositivi centrali, che garantiscono una rete priva di collisioni, e l'eventuale impiego di **hub** solo a livello locale, per connettere gruppi di dispositivi a una singola porta dello switch. Questo approccio limita i domini di collisione a piccole porzioni della rete, mantenendo la scalabilità e le alte prestazioni a livello centrale.

13 Ottimizzazione in Reti Ethernet ad Alta Velocità

L'evoluzione delle reti Ethernet ha portato a un aumento significativo della velocità di trasmissione, richiedendo l'ottimizzazione del protocollo di accesso per garantire l'efficienza e la corretta gestione delle collisioni.

13.1 Problema della Latenza e Dimensione del Frame

L'aumento della velocità di trasmissione riduce il tempo necessario per inviare un frame. Per garantire che una stazione possa rilevare una collisione mentre sta ancora trasmettendo (secondo il protocollo CSMA/CD), è necessario che il tempo di trasmissione del frame sia superiore al tempo di propagazione massimo del segnale nel cavo.

13.1.1 Calcolo del Tempo di Propagazione

Considerando una topologia a stella con hub e la massima lunghezza del cavo di 200 m per segmento, il tempo di propagazione massimo per una collisione (tempo di andata e ritorno) si calcola su una distanza totale di 800 m. (da A a HUB, da HUB a C, da C a HUB, da HUB ad A)

$$\text{Tempo di propagazione} = \frac{\text{Distanza}}{\text{Velocità di propagazione}} = \frac{800 \text{ m}}{2 \times 10^8 \text{ m/s}} = 4 \mu\text{s}$$

Con una velocità di 1 Gbps, in 4 μs si possono trasmettere:

$$1 \text{ Gbps} \times 4 \mu\text{s} = 10^9 \text{ bps} \times 4 \times 10^{-6} \text{ s} = 4000 \text{ bit} = 500 \text{ byte}$$

Per questo motivo, lo standard **Gigabit Ethernet** ha modificato la dimensione minima del frame da 64 a **512 byte** e ha ridotto la lunghezza massima del canale a 800 m, per garantire che il tempo di trasmissione superi il tempo di propagazione massimo, permettendo la corretta gestione delle collisioni.

ATTENZIONE: se si usano dispositivi vecchi (a 10 Mbit/s per esempio), in teoria continuerebbe a pubblicare solo 64 byte (troppo corto), per questo motivo se esiste la porta a un Gigabit si occupa lei di mettere un padding per raggiungere sempre i 512 byte

13.2 Il Ruolo del Livello Data Link (Livello 2)

Il livello Data Link si occupa della comunicazione tra dispositivi direttamente collegati su una rete e si divide in due sottolivelli.

- **MAC (Media Access Control):** Gestisce l'accesso al mezzo trasmissivo e il formato dei frame. In reti con mezzo condiviso (come quelle con hub), il MAC usa protocolli come il CSMA/CD. Con l'uso degli switch, il MAC gestisce canali dedicati, rendendo CSMA/CD non più necessario.
- **LLC (Logical Link Control):** Fornisce un'astrazione del livello MAC ai livelli superiori, nascondendo la complessità del mezzo condiviso e presentando la connessione come un semplice collegamento punto-punto.

14 Virtual LAN (VLAN)

Una VLAN è un'astrazione logica di una rete che consente di raggruppare dispositivi indipendentemente dalla loro posizione fisica.

14.1 Vantaggi delle VLAN

- **Sicurezza:** Le macchine in VLAN diverse sono isolate e non possono comunicare tra loro a livello di Livello 2, riducendo i rischi di accesso non autorizzato. (questo perché per comunicare fisicamente c'è il requisito in più, cioè l'id vlan)
- **Flessibilità:** I dispositivi possono essere raggruppati in base alla loro funzione, non alla loro ubicazione fisica.
- **Ottimizzazione del Traffico:** Il traffico di broadcast è confinato all'interno della propria VLAN, riducendo la congestione e migliorando le prestazioni della rete.

14.2 Funzionamento e Formato dei Frame

Gli switch supportano le VLAN attraverso una **tabella di forwarding estesa**, che associa non solo l'indirizzo MAC e la porta, ma anche un **VLAN ID**.

- **Frame Untagged:** Sono i frame inviati da un host a uno switch o viceversa. Non contengono alcuna informazione VLAN, che viene assegnata dallo switch in base alla porta di ingresso, usati per comunicazioni sulla stessa VLAN (lo switch associa il MAC al VLAN ID per vedere se sono sulla stessa rete).
- **Frame Tagged:** Sono i frame che viaggiano tra switch su un collegamento **trunk**. Vengono modificati con l'aggiunta di un **tag 802.1Q** che contiene il VLAN ID (quindi in comunicazioni tra pc su VLAN diverse) solo se esiste un disp di livello 3.

L'aggiunta del tag 802.1Q aumenta la dimensione del frame Ethernet.

14.3 Comunicazione tra VLAN

Per permettere la comunicazione tra VLAN diverse, è necessario un dispositivo che operi a **Livello 3**, come un **router** o uno **switch Layer 3**. Questi dispositivi sono in grado di instradare i pacchetti basandosi sugli indirizzi IP, superando l'isolamento di Livello 2 imposto dalle VLAN.

15 Struttura di un Pacchetto IPv4

Un pacchetto IPv4 è composto da un **header** e un **payload**. L'header standard ha una lunghezza di 20 byte, ma può essere esteso con opzioni.

15.1 Campi dell'Header IPv4

L'header IPv4 è suddiviso in numerosi campi che ne definiscono il funzionamento.

1. **Version (4 bit)**: Indica la versione del protocollo IP (4 per IPv4).
2. **Lunghezza Header (IHL) (4 bit)**: Specifica la lunghezza dell'header in parole da 32 bit.
3. **Type of Service (ToS) (8 bit)**: Definisce la priorità del pacchetto.
4. **Total Length (16 bit)**: Indica la lunghezza totale del pacchetto (header + payload) in byte.
5. **Identification (16 bit)**: Numero identificativo del pacchetto originale, utilizzato per il riassemblaggio dei frammenti.
6. **Flags (3 bit)**:
 - **D bit (Don't Fragment)**: Impedisce la frammentazione.
 - **M bit (More Fragments)**: Indica che ci sono altri frammenti successivi.
7. **Fragment Offset (13 bit)**: Indica la posizione di un frammento all'interno del pacchetto originale, in multipli di 8 byte.
8. **Time To Live (TTL) (8 bit)**: Numero massimo di "hop" (router) che il pacchetto può attraversare.
9. **Protocol Selector (8 bit)**: Indica il protocollo di livello superiore (es. TCP o UDP) del payload.
10. **Header Checksum (16 bit)**: Controllo di errore sull'header, ricalcolato a ogni hop.
11. **Source IP Address (32 bit)**: L'indirizzo IP del mittente.
12. **Destination IP Address (32 bit)**: L'indirizzo IP del destinatario.

(nel pacchetto poi ci sarebbero anche i campi options (Un campo opzionale usato per funzionalità avanzate, come il "source routing" o la registrazione del percorso.) e payload (I dati veri e propri che vengono trasportati dal pacchetto))

15.2 Gestione della Frammentazione

La frammentazione avviene a due livelli:

1. A **Livello 4** (es. TCP): Segmentazione dei dati utente in pacchetti di dimensione massima di 65.535 byte.
2. A **Livello 3** (IP): Avviene quando un pacchetto supera la **Maximum Transmission Unit (MTU)** della rete sottostante, adattando i pacchetti alla dimensione massima consentita dal livello fisico.

Il processo si basa sui campi dell'header (per il livello 3):

- Ogni frammento riceve lo stesso **Identification** del pacchetto originale.
- I **Flags** (in particolare il bit M) indicano se ci sono altri frammenti.
- Il **Fragment Offset** specifica la posizione del frammento nel pacchetto originale, usando unità di 8 byte.

Il riassemblaggio dei frammenti avviene solo sul dispositivo finale.

15.3 Esempio di Frammentazione

Consideriamo un pacchetto originale di 7000 byte che attraversa due reti con MTU differenti. Il pacchetto IP originale ha un header di 20 byte e un payload di 6980 byte.

15.3.1 Passaggio su Rete Token Ring (MTU = 4000 B)

Il pacchetto originale di 7000 B supera la MTU di 4000 B. Viene quindi frammentato in due parti, ciascuna con un nuovo header IP di 20 byte. Il payload di ogni frammento non può superare $4000 - 20 = 3980$ B, e deve essere un multiplo di 8.

- **Primo frammento:**

- Dimensione del payload: 3976 B (3976 è un multiplo di 8).
- Dimensione totale: $3976 + 20$ (header) = 3996 B.
- Fragment Offset: 0 (primo frammento).
- More Fragments (MF) bit: Impostato a 1.

- **Secondo frammento:**

- Dimensione del payload: $7000 - 20$ (header originale del frammento da 7000) $- 3976 = 3024$ B.
- Dimensione totale: $3024 + 20$ (header) = 3044 B.
- Fragment Offset: $\frac{3976}{8} = 497$.
- More Fragments (MF) bit: Impostato a 0 (ultimo frammento).

15.3.2 Passaggio su Rete Ethernet (MTU = 1500 B)

Il secondo router riceve i due frammenti precedenti e li deve frammentare ulteriormente, poiché entrambi superano la MTU di 1500 B. La dimensione massima del payload di un frammento è $1500 - 20$ (header) = 1480 B.

- **Frammentazione del primo frammento (3996 B totali / 3976 B payload):**

- **Sotto-frammento 1:** payload 1480 B, offset 0.
- **Sotto-frammento 2:** payload 1480 B, offset $\frac{1480}{8} = 185$.
- **Sotto-frammento 3:** payload $3976 - 1480 - 1480 = 1016$ B, offset $\frac{1480+1480}{8} = 370$.

- **Frammentazione del secondo frammento (3044 B totali / 3024 B payload):**

- **Sotto-frammento 4:** payload 1480 B, offset $370 + \frac{1016}{8} = 370 + 127 = 497$.
- **Sotto-frammento 5:** payload 1016 B, offset $497 + \frac{1480}{8} = 497 + 185 = 682$.
- **Sotto-frammento 6:** payload 528 B, offset $682 + \frac{1016}{8} = 682 + 127 = 809$.

16 Tecniche di Indirizzamento IP

L'indirizzamento IP è gestito dall'ICANN e dai registri regionali. Inizialmente, gli indirizzi IPv4 erano organizzati in classi.

16.1 Classi di Indirizzi IP

L'indirizzamento IPv4 è suddiviso in classi (A, B, C) in base al valore del primo ottetto. Questa classificazione determina la ripartizione tra l'identificativo di rete (**Net ID**) e quello dell'host (**Host ID**).

- **Classe A** (primo ottetto per net id il resto per host): Il primo bit è **0**. L'indirizzo è suddiviso in **7 bit per il Net ID** e **24 bit per l'Host ID**. È adatta per reti molto grandi. L'intervallo del primo ottetto è da 1 a 127.
- **Classe B** (primo e secondo ottetto per net id il resto per host): I primi due bit sono **10**. L'indirizzo è suddiviso in **14 bit per il Net ID** e **16 bit per l'Host ID**. È adatta per reti di medie dimensioni. L'intervallo del primo ottetto è da 128 a 191.
- **Classe C** (primo, secondo e terzo ottetto per net id il resto per host): I primi tre bit sono **110**. L'indirizzo è suddiviso in **21 bit per il Net ID** e **8 bit per l'Host ID**. È adatta per piccole reti. L'intervallo del primo ottetto è da 192 a 223.
- **Classe D/E**: (D)multicast per router con stesso ip, (E)riservati per usi futuri o sperimentale, entrambi non hanno parte di rete e parte host. primo ottetto da 224 a 256

16.1.1 Esempi di Suddivisione

La classe di un indirizzo IP determina quali ottetti identificano la rete e quali l'host.

- **Classe A:** L'indirizzo 10.255.255.255 è un indirizzo di Classe A. Il **Net ID** è 10, e il **Host ID** è 255.255.255. Tutti i dispositivi della rete 10.0.0.0 avranno lo stesso Net ID, mentre il resto dell'indirizzo identifica il singolo host.
- **Classe B:** Un indirizzo come 172.16.255.255 è di Classe B. Il **Net ID** è 172.16 e il **Host ID** è 255.255. Questo indirizzo identifica l'ultimo host possibile all'interno della rete 172.16.0.0.
- **Classe C:** Un indirizzo 192.168.1.255 appartiene alla Classe C. Il **Net ID** è 192.168.1 e il **Host ID** è 255.

Questo approccio facilita il routing: un router esamina solo il Net ID. Se il Net ID sorgente e quello destinazione sono uguali, il pacchetto rimane nella rete locale; se sono diversi, viene instradato verso il gateway di uscita.

16.2 Subnetting

L'approccio basato sulle classi si è rivelato inefficiente, causando lo spreco di indirizzi. Per risolvere questo problema è stato introdotto il **Subnetting**, una tecnica che permette di suddividere una rete di grandi dimensioni in sottoreti più piccole.

- Il subnetting introduce un livello gerarchico aggiuntivo, utilizzando una parte dei bit dell'Host ID per creare un identificativo di sottorete.
- La **subnet mask** (maschera di sottorete) viene usata per determinare la subnet di un host. Eseguendo un'operazione **AND bit a bit** tra l'indirizzo IP e la subnet mask, si ottiene l'indirizzo della sottorete.
- La notazione **CIDR** (es. /22) indica la lunghezza della parte di rete e sottorete dell'indirizzo. Ad esempio, '/22' significa che 22 bit (i bit a 1 nella subnet mask) sono usati per identificare la rete e la sottorete, mentre i restanti bit (a 0) identificano gli host.

Questa tecnica ha permesso di ottimizzare l'uso dello spazio degli indirizzi IPv4.

16.2.1 Esempio Pratico

Consideriamo un indirizzo IP 172.16.20.50 e una subnet mask 255.255.255.0 (notazione CIDR: /24). L'indirizzo della sottorete si ottiene applicando l'operazione **AND bit a bit** tra l'indirizzo IP e la subnet mask.

```
Indirizzo IP:  10101100.00010000.00010100.00110010
Subnet Mask:   11111111.11111111.11111111.00000000
Risultato AND: 10101100.00010000.00010100.00000000
```

Il risultato, convertito in notazione decimale puntata, è 172.16.20.0, che rappresenta l'indirizzo della sottorete. Tutti i dispositivi con un indirizzo che inizia con 172.16.20. appartengono a questa stessa sottorete.

17 CIDR (Classless Inter-Domain Routing)

Il **CIDR** (Classless Inter-Domain Routing) è stato introdotto per ottimizzare l'uso degli indirizzi IPv4 e rallentare l'esaurimento dello spazio disponibile. A differenza del precedente schema basato su classi (A, B, C), CIDR consente di assegnare blocchi di indirizzi in base alle reali necessità, riducendo lo spreco di IP e migliorando l'efficienza del routing.

17.1 Struttura e Assegnazione CIDR

Con CIDR, gli indirizzi IP vengono assegnati in blocchi a dimensione variabile, identificati da una notazione speciale: IP_ADDRESS/SUBNET_MASK.

Dove SUBNET_MASK rappresenta il numero di bit utilizzati per identificare la rete. Ad esempio, 192.168.1.0/24 indica che i primi 24 bit sono riservati per la rete, lasciando gli ultimi 8 bit per gli host (2^8 indirizzi possibili).

17.2 Esempi di Allocazione e Calcolo CIDR

Analizziamo alcuni esempi per comprendere come CIDR ottimizza l'allocazione e la gestione degli indirizzi IP.

17.2.1 Esempio 1: Calcolo della Maschera CIDR

Domanda: Un'azienda necessita di 4000 indirizzi IP. Qual è la maschera CIDR minima necessaria?

Risposta:

- Il numero di host richiesti è 4000. Il numero più vicino in potenza di 2 è 4096 (2^{12}).
- Sono necessari 12 bit per gli host. Poiché un indirizzo IPv4 è composto da 32 bit, la maschera CIDR si calcola come $32 - 12 = 20$.
- La subnet mask minima necessaria è /20.

17.2.2 Esempio 2: Primo e Ultimo Indirizzo Disponibile

Domanda: Dato il blocco 10.10.32.0/20, quali sono il primo e l'ultimo indirizzo disponibile?

Risposta:

- La maschera /20 lascia $32 - 20 = 12$ bit per gli host, quindi il numero di indirizzi è $2^{12} = 4096$.
- L'intervallo è da 10.10.32.0 a 10.10.47.255.
- Il primo indirizzo utilizzabile è 10.10.32.1.
- L'ultimo indirizzo utilizzabile è 10.10.47.254.

17.2.3 Esempio 3: Appartenenza a una Rete

Domanda: Un ISP assegna l'intervallo 172.16.64.0/22. L'indirizzo 172.16.66.25 appartiene a questa rete?

Risposta:

- La maschera /22 lascia $32 - 22 = 10$ bit per gli host. Il range degli indirizzi va da 172.16.64.0 a 172.16.67.255.
- L'indirizzo 172.16.66.25 è compreso in questo intervallo, quindi sì, l'IP appartiene alla subnet.

CIDR ha prolungato la vita di IPv4, ma non l'ha resa eterna, richiedendo l'adozione di altre tecnologie come il NAT per la sua gestione.

18 NAT (Network Address Translation)

L'adozione di NAT (Network Address Translation) ha contribuito significativamente a prolungare la vita di IPv4, pur non rendendolo eterno. Molte reti continuano a funzionare con IPv4 grazie a NAT, che consente una gestione ottimizzata degli indirizzi IP.

18.1 Cos'è il NAT e a cosa serve?

NAT è una tecnologia che permette di tradurre gli indirizzi IP privati in un indirizzo IP pubblico e viceversa. Questo meccanismo consente di ottimizzare l'uso degli indirizzi IPv4 disponibili, proteggere la rete privata dall'accesso diretto all'esterno e separare lo spazio degli indirizzi pubblici da quello privato.

18.2 Struttura del NAT

In una rete privata, è possibile assegnare ai dispositivi indirizzi IP privati, che non sono visibili su Internet. Un dispositivo NAT, tipicamente un router, possiede un unico indirizzo IP pubblico e funge da intermediario tra la rete privata e il mondo esterno. Gli indirizzi IP privati utilizzabili in una rete interna appartengono ai seguenti intervalli riservati:

- 10.0.0.0/8
- 172.16.0.0/12
- 192.168.0.0/16

Nota: Questi indirizzi possono essere riutilizzati in più reti senza causare conflitti, poiché il traffico di rete privata non è instradabile su Internet.

18.3 Funzionamento del NAT

Quando un dispositivo all'interno della rete privata invia una richiesta a un server su Internet:

1. Il pacchetto viene intercettato dal NAT.
2. Il NAT sostituisce l'indirizzo IP sorgente del dispositivo privato con il proprio indirizzo IP pubblico.
3. Il pacchetto viene inoltrato al server di destinazione.

Alla ricezione della risposta da parte del server:

1. Il NAT intercetta il pacchetto in ingresso.
2. Utilizzando una tabella di associazione, il NAT sostituisce l'indirizzo IP di destinazione con quello del dispositivo privato corrispondente.
3. Il pacchetto viene inoltrato al dispositivo all'interno della rete privata.

18.4 Problema della gestione delle risposte

Il NAT tradizionale permette di inviare richieste all'esterno, ma inizialmente non gestisce correttamente le risposte perché i dispositivi interni utilizzano indirizzi IP privati, che non sono direttamente raggiungibili da Internet. Quando un server su Internet invia una risposta, essa è indirizzata all'IP pubblico del NAT, che però non può sapere a quale dispositivo interno restituire il pacchetto senza un meccanismo di associazione. Per risolvere questa limitazione, si introduce una tabella di associazione con i seguenti campi.

Quando un dispositivo interno invia una richiesta:

- Il NAT registra nella tabella l'indirizzo IP privato, la porta sorgente e il corrispondente indirizzo pubblico con una porta temporanea univoca (**Port Address Translation - PAT**).
- Quando la risposta arriva all'indirizzo pubblico e alla porta assegnata, il NAT consulta la tabella e reindirizza il pacchetto al dispositivo interno corretto.

18.5 Violazione del modello OSI: il NAT e la manipolazione dei livelli

Il NAT introduce un'anomalia rispetto alla struttura a livelli del modello OSI:

- Il modello OSI prevede una chiara separazione tra i livelli (ad es. il livello di rete non dovrebbe interagire con il livello di trasporto).
- Il NAT, tuttavia, deve modificare informazioni sia nel livello 3 (IP) che nel livello 4 (TCP/UDP), sostituendo non solo gli indirizzi IP, ma anche i numeri di porta.
- Questa "sporcizia" architetturale è necessaria per consentire il funzionamento del NAT, anche se rompe il principio di astrazione tra i livelli.

18.6 Lavoro del NAT su ogni pacchetto

Ogni pacchetto che passa attraverso il NAT subisce le seguenti modifiche:

1. **Livello IP (Livello 3):** Il NAT sostituisce l'indirizzo IP sorgente con l'IP pubblico della rete.
2. **Livello di Trasporto (Livello 4 - TCP/UDP):** Il NAT sostituisce il numero di porta sorgente con un numero univoco scelto dal router.
3. **Tabella di mappatura:** Il NAT registra l'associazione tra IP e porta privata e IP e porta pubblica.

Avvertenza: Il numero di porta utilizzato da un dispositivo privato potrebbe non essere univoco tra reti diverse. Il NAT assegna dinamicamente una porta NAT univoca. Questo processo permette di mantenere le connessioni attive e garantisce il corretto instradamento dei pacchetti di risposta.

18.7 NAT e server accessibili dall'esterno

Se si desidera esporre un server interno su Internet, esistono due soluzioni:

1. Assegnare un indirizzo IP pubblico statico al server, rendendolo direttamente accessibile.
2. Utilizzare il **Port Forwarding** (o **Destination NAT - DNAT**): il NAT inoltra il traffico in arrivo su una specifica porta dell'IP pubblico a un IP privato interno.

Esempio: per rendere un server web accessibile dall'esterno, si configura il NAT affinché tutto il traffico destinato all'IP pubblico sulla porta 80 venga inoltrato all'IP privato del server web.

19 ARP (Address Resolution Protocol) (come trovare il mac di un dispositivo se non lo conosco), (arp si fa solo nelle reti locali dato che il MAC serve per trasmettere l'informazione nel canale fisico ip nel logico)

L'**Address Resolution Protocol (ARP)** è un protocollo di livello Network (Livello 3 OSI) che consente di determinare l'indirizzo MAC (Media Access Control, livello 2) associato a un dato indirizzo IP. Quando un dispositivo in una rete locale (LAN) deve inviare un pacchetto a un altro dispositivo, conosce il suo indirizzo IP ma ha bisogno dell'indirizzo MAC per poter effettivamente inviare il frame Ethernet.

19.1 Come risolve un IP in un MAC address?

Il dispositivo sorgente invia una richiesta ARP (**ARP Request**) in **broadcast** sulla rete, chiedendo "Chi ha l'indirizzo IP X.X.X.X? Rispondi con il tuo indirizzo MAC". Il dispositivo destinatario, se presente nella LAN, risponde con un messaggio **ARP Reply** in **unicast**, fornendo il proprio MAC address. Il mittente aggiorna la propria **tabella ARP** con l'associazione IP-MAC ricevuta, in modo da poter inviare direttamente i successivi pacchetti senza dover ripetere la risoluzione ARP.

19.2 Funzionamento dettagliato di ARP

Ogni macchina di una LAN ha un modulo ARP che si occupa della gestione delle richieste e delle risposte.

1. Verifica della cache ARP

- Prima di inviare un pacchetto, il dispositivo controlla se ha già il MAC address dell'IP di destinazione nella sua cache ARP.
- Se l'entry esiste ed è valida, il dispositivo può inviare direttamente il frame.
- Se non esiste, o è scaduta, si procede con una ARP Request.

2. Invio dell'ARP Request

- Il dispositivo sorgente crea un pacchetto ARP Request e lo invia in broadcast (MAC di destinazione: FF:FF:FF:FF:FF:FF).
- Il pacchetto contiene IP e MAC mittente e IP del destinatario.

3. Risposta con ARP Reply

- Il dispositivo con l'IP richiesto riceve l'ARP Request e risponde con un ARP Reply in unicast, includendo il proprio MAC address.
- Il mittente aggiorna la sua cache ARP e utilizza l'indirizzo MAC ricevuto per la trasmissione dei pacchetti successivi.

4. Cache ARP e TTL

- Le associazioni IP-MAC apprese vengono memorizzate nella cache ARP per un certo periodo di tempo (**TTL** – Time to Live).
- Questo evita di dover ripetere continuamente richieste ARP per dispositivi con cui si comunica frequentemente.

19.3 Formato del pacchetto ARP

Un pacchetto ARP è incapsulato direttamente in un frame Ethernet, con **EtherType** 0x0806 (ARP).

19.4 ARP su reti diverse: il ruolo del router (Gateway)

Attenzione: Quando la destinazione è all'esterno della LAN, una ARP Request tradizionale non riceverebbe risposta, perché i pacchetti broadcast non vengono inoltrati tra reti diverse.

19.4.1 Come si risolve il problema?

1. Calcolo del Net ID

- La macchina esegue un **AND bit a bit** tra il proprio IP e la Subnet Mask, ottenendo il suo Net ID.
- Fa lo stesso con l'IP di destinazione.

2. Confronto dei Net ID

- Se i Net ID coincidono: la destinazione è nella stessa LAN, può inviare il pacchetto direttamente.
- Se i Net ID sono diversi: la destinazione è fuori dalla LAN, quindi il pacchetto deve passare attraverso il gateway (router).

3. Entra in gioco il gateway

- Se i Net ID sono diversi, il pacchetto non può essere inviato direttamente al destinatario.
- Invece di fare un'ARP Request per la destinazione, la macchina sorgente fa un'ARP Request per il **gateway predefinito**.

4. Risoluzione dell'IP del gateway in un MAC

- Se il MAC del gateway predefinito è già noto (cache ARP), il pacchetto viene subito incapsulato in una trama Ethernet e inviato al router.
- Se il MAC del gateway predefinito non è noto, il dispositivo invia una ARP Request chiedendo: "Chi ha l'IP del gateway? Rispondi con il tuo MAC!"
- Il router risponde con una ARP Reply, fornendo il suo MAC.

5. Invio del pacchetto al router

- Il dispositivo sorgente incapsula il pacchetto IP in un frame Ethernet, usando:
 - MAC sorgente = suo MAC
 - MAC destinazione = MAC del gateway
- Il router riceve il pacchetto e si occupa di inoltrarlo verso la rete di destinazione. (in modalità proxy, l'unica cosa che cambia è che non serve più che la macchina capisca da sola che deve fare la richiesta per il MAC del router, ma è già il router che capisce che deve fingersi il MAC del dispositivo)

19.4.2 Proxy ARP

Esiste un meccanismo chiamato **Proxy ARP**, usato quando un dispositivo deve raggiungere una destinazione fuori dalla LAN, ma non sa che deve passare per un router.

- Il router, vedendo l'ARP Request per un IP esterno, risponde con il proprio MAC address, anche se l'IP richiesto appartiene a un'altra rete.
- In questo modo, il mittente crede che l'IP di destinazione sia nella stessa LAN e invia i pacchetti al router, che poi li inoltra correttamente.
- Proxy ARP è meno utilizzato oggi a causa della separazione più chiara tra reti e della diffusione del routing statico e dinamico.

20 DHCP, ICMP e Routing

20.1 Rinnovo e Rilascio dell'IP

Gli indirizzi IP assegnati dal DHCP hanno un tempo di lease (validità). Il client tenta di rinnovare il lease in momenti specifici per evitare di perdere l'indirizzo. Se il lease scade, il client deve ripetere l'intero processo **DORA** per ottenerne uno nuovo.

20.2 Formato ICMP

ICMP (Internet Control Message Protocol) è un protocollo di livello 3 che viene usato per la diagnostica e la gestione degli errori. Ad esempio, un server DHCP usa un pacchetto ICMP (*ping*) per verificare che un indirizzo IP non sia già in uso prima di assegnarlo. ICMP ha una *checksum* anche per il payload, a differenza del protocollo IP che ne ha una solo per l'header.

20.3 Routing vs. Forwarding vs. ARP

- **Routing:** Il processo di **popolare dinamicamente** la tabella di routing tramite protocolli specifici (es. RIP, OSPF), apprendendo la topologia della rete. È un processo logico e di alto livello.
- **Forwarding:** L'azione fisica di **inoltrare** un pacchetto da un'interfaccia a un'altra in base alle informazioni già presenti nella tabella di routing.
- **ARP:** trovare gli indirizzi MAC dato l'ip (il routing serve invece per trovare la strada migliore)

ATTENZIONE: per inviare un pacchetto serve sapere: **ip, MAC, porta del dispositivo**, se si avesse solo il MAC, nel momento in cui quel dispositivo diventa una sottorete, dovrei sapere tutti i mac di tutti i dispositivi e non solo l'indirizzo ip, inoltre c'è una logica diversa tra MAC e IP.

A differenza del forwarding di livello 2, che si basa sull'adiacenza fisica (MAC), il routing di livello 3 si occupa dell'instradamento tra reti diverse.

21 Protocolli di Routing a Vettore di Distanza

21.1 Principi di Funzionamento

I protocolli a vettore di distanza (come **RIP**) si basano sull'idea che ogni nodo mantenga una tabella di routing aggiornando la propria conoscenza della rete solo tramite le informazioni ricevute dai nodi vicini. Ogni nodo periodicamente:

1. Costruisce la propria tabella delle adiacenze basata sui collegamenti diretti.
2. Trasmette il proprio *distance vector* (una lista di destinazioni e costi) ai vicini.
3. Utilizza i *distance vector* dei vicini per aggiornare la propria tabella.

Questo meccanismo consente a un nodo di passare da una conoscenza locale a una *conoscenza globale* della rete, sebbene in modo graduale. La tabella di routing di un nodo conterrà:

- **Destinazione** (indirizzo di rete o host).
- **Next-hop** (il router successivo).
- **Metrica** (il costo o la distanza).

Nota: "Conoscenza globale" non significa che il nodo ha una mappa completa della rete; significa che sa raggiungere ogni destinazione, affidandosi ai suoi vicini per il percorso corretto.

21.2 RIP (Routing Information Protocol)

RIP è un protocollo basato su Vettore di Distanza che utilizza il numero di **hop** come metrica. Il suo limite massimo di hop è 15, rendendolo adatto solo per reti di piccole dimensioni.

21.3 Problemi dei Protocolli a Vettore di Distanza

L'aggiornamento asincrono dei *distance vector* può portare a problemi di convergenza.

21.3.1 Bouncing Effect

Si verifica quando i nodi continuano a scambiarsi informazioni obsolete a causa di ritardi. I costi rimbalzano tra valori diversi, portando a una propagazione errata che può causare il problema di *Count to Infinity*.

21.3.2 Count to Infinity

Il *Count to Infinity* si verifica quando una destinazione diventa irraggiungibile, ma i nodi continuano ad aggiornare il costo in modo incrementale, credendo erroneamente di poterla raggiungere tramite un vicino, fino a raggiungere un valore massimo (16 in RIP, che indica irraggiungibilità).

21.4 Soluzioni e Correzioni

1. **Triggered Updates:** Un meccanismo che invia un aggiornamento immediato non appena viene rilevato un cambiamento critico (es. un guasto), velocizzando la convergenza.
2. **Split Horizon:** Una tecnica che evita di pubblicizzare una rotta a un vicino se la rotta è stata appresa da quel vicino. Questa regola previene i loop di routing. Il router C non pubblicherà a B la sua rotta verso A.
3. **Split Horizon con Poison Reverse:** Una variante che, in caso di guasto, pubblicizza la rotta (appresa da un vicino) con un costo "infinito", per prevenire che il vicino la utilizzi erroneamente.

21.4.1 Limiti delle soluzioni: perché Triggered Updates e Split Horizon possono fallire

Anche con queste correzioni, i protocolli a vettore di distanza possono fallire in scenari specifici, ad esempio in caso di perdita di pacchetti di aggiornamento.

Scenario di fallimento:

1. **Guasto del link:** Un link tra due router (es. B e C) si interrompe.
2. **Triggered Update perso:** Il router B rileva il guasto e invia un Triggered Update ai suoi vicini (A e D) per informarli che C è irraggiungibile (costo infinito). Supponiamo che il messaggio di aggiornamento inviato a D vada perso.
3. **Propagazione di informazione errata:**
 - Il router D, non avendo ricevuto l'aggiornamento, continua a credere di poter raggiungere C attraverso B e lo comunica ai suoi vicini (es. E).
 - Il router E riceve l'informazione errata da D e la propaga a sua volta.
4. **Innesco del Count to Infinity:** L'informazione errata rientra in circolo, raggiungendo anche il router B, che "impara" una nuova rotta (sbagliata) verso C. Il costo per raggiungere C inizia ad aumentare ad ogni scambio di informazioni, fino a raggiungere il limite massimo (16 per RIP), innescando il problema del *Count to Infinity*.

Questo esempio dimostra che la perdita di un singolo aggiornamento può vanificare le contromisure e portare a gravi errori di routing, rendendo questi protocolli inaffidabili per reti complesse.

22 Limiti e Applicazioni

22.1 Inapplicabilità su Larga Scala

I protocolli a vettore di distanza come RIP sono **impossibili** da usare per il routing su larga scala (es. Internet) per tre motivi principali:

- **Scalabilità:** La tabella di routing dovrebbe contenere un'entrata per ogni rete globale, un numero impraticabile.
- **Convergenza Lenta:** I guasti impiegherebbero troppo tempo a propagarsi, causando inefficienze diffuse.
- **Consumo di Banda:** L'invio periodico di tabelle di routing complete consumerebbe una quantità enorme di banda.

Internet utilizza invece protocolli più avanzati e scalabili come **BGP** (Border Gateway Protocol), che si basa sul concetto di sistemi autonomi.

23 Protocolli di Routing Link State

Per superare i limiti dei protocolli a vettore di distanza, le reti moderne utilizzano i protocolli **Link State** (es. OSPF), che offrono una convergenza più rapida e una maggiore affidabilità.

23.1 Principi di Funzionamento (Link State)

In un protocollo Link State, ogni nodo costruisce una mappa completa della topologia di rete attraverso questi passaggi:

1. **Scoperta dei vicini:** Ogni router utilizza pacchetti 'HELLO' per identificare i router direttamente connessi (adiacenze).
2. **Propagazione dello stato del link:** Ogni router crea un pacchetto chiamato **Link State Advertisement (LSA)** che descrive i suoi collegamenti diretti e i relativi costi. Questo LSA viene inviato a tutti gli altri router della rete tramite un processo di *flooding controllato*.
3. **Costruzione della mappa topologica:** Ogni router raccoglie tutti gli LSA ricevuti dagli altri router e li utilizza per costruire una mappa completa e identica della rete.
4. **Calcolo del cammino minimo:** Una volta ottenuta la mappa, ogni router esegue in modo indipendente un algoritmo di cammino minimo, come l'algoritmo di **Dijkstra**, per calcolare il percorso migliore (a costo più basso) verso ogni possibile destinazione.

23.2 Vantaggi e Svantaggi dei Protocolli Link State

- **Vantaggi:**
 - **Visione completa della topologia:** Elimina i loop di routing e il problema del *Count to Infinity*.
 - **Convergenza rapida:** Le informazioni sui cambiamenti di topologia vengono propagate rapidamente tramite flooding, riducendo i tempi di riconfigurazione.
 - **Selezione del percorso ottimale:** L'uso dell'algoritmo di Dijkstra garantisce il calcolo del percorso migliore basato sulla metrica definita.
- **Svantaggi (Costi):**
 - **Overhead di rete:** Il flooding degli LSA può generare un traffico di controllo significativo, specialmente in reti di grandi dimensioni.
 - **Carico computazionale:** L'esecuzione dell'algoritmo di Dijkstra richiede più risorse di CPU e memoria rispetto ai protocolli a vettore di distanza.

23.3 OSPF (Open Shortest Path First)

OSPF è il protocollo Link State più diffuso. Le sue caratteristiche principali includono:

- **Struttura gerarchica:** La rete viene suddivisa in **Aree** per ridurre il traffico di aggiornamento e la complessità di calcolo.
- **Metriche avanzate:** Il costo di un link può essere basato su metriche come larghezza di banda e latenza, non solo sul numero di hop.
- **Bilanciamento del carico (Multipath):** Se esistono più percorsi con lo stesso costo verso una destinazione, OSPF può distribuire il traffico su di essi.

23.3.1 Struttura di una Rete OSPF: Le Aree

Una rete OSPF è organizzata in modo gerarchico con una struttura a stella.

- **Area 0 (Backbone Area):** È l'area centrale che funge da "spina dorsale" della rete.
- **Altre Aree:** Tutte le altre aree devono essere collegate direttamente all'Area 0. La comunicazione tra due aree non-backbone deve sempre passare attraverso l'Area 0.

Questa struttura limita il flooding degli LSA all'interno di una singola area, migliorando la scalabilità.

23.3.2 Ottimizzazioni in OSPF: Designated Router (DR)

Per ridurre il traffico di controllo nelle reti broadcast (come Ethernet), OSPF elegge un **Designated Router (DR)** e un **Backup Designated Router (BDR)**.

- **Ruolo del DR:** Invece di scambiare LSA con tutti, i router inviano i loro aggiornamenti solo al DR. Sarà poi il DR a distribuire le informazioni a tutti gli altri router del segmento.
- **Vantaggio:** Questo meccanismo riduce drasticamente il numero di adiacenze e il traffico di flooding.

24 Routing su Larga Scala e Architetture Moderne

24.1 Routing tra Sistemi Autonomi: BGP

Il routing su Internet è basato sul concetto di **Autonomous System (AS)**: una grande rete sotto un'unica amministrazione (es. un ISP).

- **Protocolli IGP (Interior Gateway Protocol):** Come OSPF e RIP, gestiscono il routing *all'interno* di un singolo AS.
- **Protocolli EGP (Exterior Gateway Protocol):** Come **BGP (Border Gateway Protocol)**, gestiscono il routing *tra* diversi AS. (BGP è un protocollo distance vector che però aggiunge anche la lista degli hop che il percorso ha effettuato, questo elimina il problema del bouncing effect filtrando i loop sulle varie stazioni (PATH VECTOR))

I router ai confini di un AS, chiamati **Border Router**, utilizzano BGP per scambiare informazioni di raggiungibilità con altri AS, permettendo la comunicazione su scala globale.

ATTENZIONE: Un Sistema Autonomo (AS) è una rete gestita da un'unica entità, suddivisa in aree OSPF per motivi di scalabilità. Al suo interno, un protocollo IGP (come OSPF) gestisce il routing: l'Area 0 (backbone) collega le altre aree periferiche (come Area 1, 2, ecc.) che, a loro volta, contengono le reti locali. I router che collegano le aree periferiche all'Area 0 (chiamati ABR) usano OSPF. Un diverso protocollo, EGP (come BGP), viene usato dai router di confine dell'AS per collegare l'intero sistema ad altri AS, formando la spina dorsale di Internet.

24.2 Evoluzione: Software Defined Networking (SDN)

L'approccio tradizionale di routing distribuito è stato superato in molte reti moderne dal paradigma **SDN (Software Defined Networking)**.

- **Separazione tra Control Plane e Data Plane:** SDN separa la logica di routing (il "cervello", o Control Plane) dall'azione fisica di inoltrare i pacchetti (il "braccio", o Data Plane).
- **Controller Centralizzato:** La logica di routing viene spostata dai singoli router a un **controller SDN** centralizzato (spesso nel cloud).
- **Funzionamento:**
 1. I router inviano informazioni sullo stato della rete al controller.
 2. Il controller calcola le tabelle di routing ottimali per l'intera rete.
 3. Il controller invia le tabelle di routing ai router, che si limitano a inoltrare i pacchetti secondo le istruzioni ricevute.

SDN permette una gestione più efficiente, flessibile e centralizzata della rete, ed è fondamentale per tecnologie come il 5G.

24.3 Tecniche di Trasporto: Tunneling

Il **tunneling** è una tecnica che permette di trasportare pacchetti di un protocollo attraverso una rete che ne utilizza un altro.

1. **Incapsulamento:** Il pacchetto originale (es. IPv6) viene "avvolto" dentro un altro pacchetto con un header compatibile con la rete di transito (es. IPv4).
2. **Trasporto:** Il pacchetto incapsulato viaggia attraverso la rete di transito.
3. **Decapsulamento:** A destinazione, l'header esterno viene rimosso e il pacchetto originale viene ripristinato.

Questa tecnica è essenziale per garantire la connettività tra reti con protocolli diversi o per creare reti private virtuali (VPN).

25 Border Gateway Protocol (BGP)

Il Border Gateway Protocol (BGP) è il protocollo di routing utilizzato per la comunicazione tra i router che gestiscono il flusso di traffico su Internet. Si tratta di un protocollo di livello 3 (network layer) che si occupa di addressing e routing, ma la sua implementazione avviene a livello applicativo e si appoggia su TCP per garantire una comunicazione affidabile.

Caratteristiche principali

- BGP utilizza il protocollo TCP sulla porta 179 per stabilire connessioni affidabili tra i router.
- Mantiene sessioni permanenti di TCP tra le unità BGP per scambiare informazioni di routing in modo stabile e sicuro.
- È utilizzato principalmente per connettere diversi Autonomous Systems (AS) all'interno della backbone di Internet, una rete globale e distribuita a livello intercontinentale.

Gestione delle rotte e edel percorso ottimale BGP si occupa della selezione del percorso ottimale per la trasmissione dei pacchetti. A differenza di altri protocolli di routing, non utilizza algoritmi di flooding, ma si basa su una variante del Distance Vector Routing chiamata Path Vector Routing.

25.1 Path Vector Routing e gestione del Bouncing Effect

Invece di trasmettere semplicemente i costi delle rotte, come nei protocolli Distance Vector tradizionali (ad esempio RIP), BGP include informazioni aggiuntive sul percorso, note come Path Attributes. Ogni nodo BGP propaga un vettore di cammini contenente l'elenco degli Autonomous Systems attraversati per raggiungere una determinata destinazione. Questo meccanismo previene il "bouncing effect" (routing loops), poiché un AS può rifiutare percorsi che contengono se stesso nella lista degli AS attraversati, evitando così di riutilizzare un link guasto.

Important Se un link si guasta, BGP seleziona un nuovo percorso alternativo, anche se ha un costo maggiore, garantendo così la connettività.

26 Problematiche del routing tradizionale

Nei router, indipendentemente dal fatto che abbiano o meno una funzione di routing interna, esistono operazioni che richiedono l'elaborazione dell'header IP, in particolare degli indirizzi di origine (source address) e destinazione (destination address). Queste operazioni comprendono il lookup tabellare per determinare il prossimo hop del pacchetto. Nei punti critici della rete, dove sono richieste prestazioni elevate, il forwarding dei pacchetti deve essere estremamente efficiente. Tuttavia, il classico lookup tabellare basato su indirizzi IP può risultare computazionalmente oneroso. Per accelerare il processo, è necessario intervenire a livello algoritmico riducendo al minimo l'elaborazione necessaria per determinare il percorso di un pacchetto.

27 MPLS: Multi-Protocol Label Switching

Per migliorare il forwarding nella backbone area, è stato introdotto MPLS (Multi-Protocol Label Switching), una tecnologia che permette di velocizzare il processo eliminando la necessità di lookup basati su indirizzi IP. Di seguito le sue caratteristiche:

- *Switching anziché Routing*: MPLS adotta una logica di switching, tipicamente associata al livello 2 del modello OSI, invece di operare un forwarding basato su IP (livello 3).
- *Uso di Etichette (Labels)*: Invece di analizzare l'intero indirizzo IP di destinazione, MPLS utilizza semplici etichette numeriche per determinare il percorso del pacchetto.
- *Riduzione dell'Overhead Computazionale*: Poiché i router non devono eseguire un complesso lookup tabellare su un indirizzo IP, il forwarding dei pacchetti avviene in modo significativamente più rapido.

28 Funzionamento del MPLS

Il meccanismo di MPLS può essere suddiviso nelle seguenti fasi:

1. Etichettatura del Traffico:

- I pacchetti provenienti dalle aree periferiche e diretti verso la backbone area vengono marcati con un'etichetta MPLS.
- Questa etichetta numerica viene assegnata secondo le policy definite dall'operatore di rete.
- I router di confine della backbone area, noti come ABR (Area Border Routers), sono responsabili dell'aggiunta dell'header MPLS (etichetta) ai pacchetti in ingresso

2. Forwarding Basato su Etichetta:

- All'interno della backbone area, i router non eseguono un classico routing basato su IP, ma effettuano lo switching dei pacchetti basandosi esclusivamente sulle etichette MPLS.
- Questo consente di evitare lookup IP complessi e velocizza il processo di inoltramento.

3. Decapsulazione in Uscita:

- Quando il pacchetto raggiunge il router di bordo della destinazione finale, l'ABR di uscita rimuove l'header MPLS e ripristina il normale header IP.
- Il pacchetto originale viene quindi inoltrato utilizzando il normale routing IP fino alla sua destinazione finale.

29 Gestione delle etichette

Ogni router assegna e gestisce le etichette in modo autonomo. È possibile etichettare non solo i singoli pacchetti, ma anche interi flussi di traffico.

Note Ad esempio, tutto il traffico che viaggia dall'aula Alpha all'aula Beta può essere contrassegnato con la stessa etichetta, permettendo una gestione più efficiente. L'elemento chiave che omogeneizza questi flussi è la Qualità del Servizio (QoS), strettamente legata al campo Type of Service (ToS) presente nei pacchetti IP

30 Costruzione delle tabelle di switching

Le tabelle di switching vengono costruite in modo da ottimizzare i percorsi dei pacchetti con la stessa etichetta, garantendo il miglior instradamento possibile. In questo contesto, il protocollo OSPF non viene eliminato, ma continua a svolgere un ruolo fondamentale:

- Deve raccogliere le informazioni sullo stato dei collegamenti tra i router nell'area di rete.
- Deve costruire la topologia del grafo della rete per identificare i cammini minimi.

Invece di propagare tradizionali tabelle di routing, il Designated Router si occupa della distribuzione delle tabelle di switching, facilitando l'instradamento basato sulle etichette. Questo approccio, noto come Label Switching, permette di velocizzare il traffico di rete rispetto al tradizionale IP Routing, migliorando l'efficienza e riducendo il carico computazionale sui router.

31 Formato del pacchetto MPLS

Un pacchetto MPLS è strutturato nel seguente modo: Di seguito la composizione di un pacchetto MPLS: Di seguito, la descrizione dei principali campi dell'header MPLS:

- *Label*: Funziona da identificatore e permette l'instradamento del pacchetto basato su etichette anziché sugli indirizzi IP tradizionali.
- *Class of Service (CoS)*: Permette di classificare il pacchetto in base alla qualità del servizio (QoS). Questo consente di far fluire il pacchetto e tutti quelli appartenenti allo stesso flusso lungo un percorso specifico, garantendo priorità e gestione ottimale del traffico.
- *S (Bottom of Stack)*: Indica se il pacchetto MPLS fa parte di una pila di etichette (Multiple Label). Se il valore è 1, il pacchetto rappresenta l'ultima etichetta nella pila.
- *TTL (Time To Live)*: Serve a prevenire cicli infiniti nella rete. Ogni router decrementa il valore TTL del pacchetto, e se questo raggiunge 0, il pacchetto viene scartato.

32 Spanning tree

Lo Spanning Tree è una tecnica utilizzata nella costruzione dell'albero dei cammini minimi all'interno di una rete complessa. Viene impiegato in particolare nelle reti con protocollo Link- State, dove ogni nodo è in grado di costruire il grafo della rete e calcolare i cammini minimi verso gli altri nodi.

Tipologie di porte nello spanning tree

1. *Root Port (RP)*: Porta che punta verso la radice dell'albero.
2. *Designated Port (DP)*: Porta che conduce verso i nodi foglia.
3. *Blocked Port (BP)*: Porta disabilitata per prevenire la formazione di cicli all'interno della rete.

Funzionamento Alla ricezione di un pacchetto sulla Root Port, il router lo propaga su tutte le Designated Ports. Questo processo consente di costruire un albero dei cammini minimi, che ottimizza il numero di pacchetti inviati in una trasmissione broadcast, riducendo così la congestione della rete. L'algoritmo dello Spanning Tree è progettato per risolvere due problemi principali nelle reti di computer:

- *Evitare il sovraccarico della rete*: Previene la trasmissione ridondante di pacchetti ai nodi vicini, ottimizzando l'instradamento.
- *Prevenire cicli infiniti*: Impedisce che un pacchetto rimanga intrappolato in un loop tra due o più nodi, garantendo così un'efficiente gestione del traffico

33 Quality Of Service (QoS) nelle reti di comunicazione

Le reti tradizionali operano con un approccio **best-effort**, senza offrire garanzie sulla qualità della trasmissione. Per superare questo limite, si introduce il concetto di **Quality of Service (QoS)**, che permette di gestire e differenziare il traffico di rete in base alle esigenze specifiche delle applicazioni.

Le principali esigenze da soddisfare sono:

- **Sensibilità al ritardo (Latency)**: Tempo minimo per la trasmissione, cruciale per applicazioni real-time (es. VoIP, gaming online).
- **Banda richiesta (Bandwidth)**: Quantità di dati trasmissibile nell'unità di tempo, fondamentale per lo streaming video ad alta definizione.
- **Sensibilità al Jitter**: Variazione del ritardo di arrivo dei pacchetti. Un jitter elevato degrada la qualità di audio e video.

Per mitigare il jitter, applicazioni come lo streaming usano un **buffer di playout**, che immagazzina i dati in arrivo prima di riprodurli, garantendo un flusso costante.

33.1 Gestione delle code: Code di Ingresso (Input Queuing)

La congestione si verifica quando il traffico in ingresso a un router supera la sua capacità di elaborazione, portando alla perdita di pacchetti (**buffer overflow**). Per gestire questo problema si usano tecniche di gestione attiva delle code.

33.1.1 Random Early Detection (RED)

RED è un algoritmo di gestione proattiva della congestione che scarta i pacchetti *prima* che la coda si saturi completamente.

- **Principio di funzionamento**: Calcola un livello medio di occupazione della coda (u).
- **Soglie di funzionamento**:
 - **Soglia minima**: Sotto questa soglia, nessun pacchetto viene scartato.
 - **Soglia massima**: Sopra questa soglia, tutti i pacchetti in arrivo vengono scartati.
 - **Tra le due soglie**: I pacchetti vengono scartati con una probabilità che aumenta linearmente con l'aumentare dell'occupazione della coda.
- **Vantaggi (componente "Early")**:
 - Previene la saturazione completa del buffer.
 - Segnala in anticipo la congestione ai protocolli di trasporto (es. TCP), che possono ridurre la loro velocità di trasmissione.
 - Evita il blocco totale della rete.

33.1.2 ICMP Choke Packets

Un'altra tecnica (meno usata) prevede l'invio di pacchetti ICMP (**choke packets**) a ritroso verso la sorgente per segnalare la congestione. Presenta però degli svantaggi, come l'aumento del traffico di controllo e il ritardo di reazione.

33.2 Gestione delle code: Code di Uscita (Output Queuing)

Nella gestione delle code di uscita, un **classificatore** smista i pacchetti in arrivo in diverse code basandosi sul loro campo *Type of Service (ToS)*. Uno **scheduler** decide poi quale coda servire per la trasmissione.

33.2.1 Il Problema della Starvation

Un semplice scheduler a priorità rigida (che serve sempre la coda a priorità più alta) può portare alla **starvation**: le code a bassa priorità potrebbero non essere mai servite se c'è un flusso costante di traffico ad alta priorità.

33.2.2 Weighted Fair Queuing (WFQ) come soluzione

Per evitare la starvation, si usa il **Weighted Fair Queuing (WFQ)**. A ogni coda viene assegnato un **peso** (w_i), e lo scheduler alloca la banda in modo proporzionale a tali pesi.

- La frazione di banda per la coda i è calcolata come: “math

-

$$\frac{w_i}{\sum_j w_j}$$

- In questo modo, nessuna coda viene completamente esclusa (*starvation-free*).

Esempio: Con 3 code di pesi $w_1 = 4, w_2 = 2, w_3 = 2$, la banda viene ripartita al 50%, 25% e 25% rispettivamente.

33.2.3 Weighted RED (WRED)

Combinando WFQ e RED si ottiene **WRED**. Questa tecnica applica soglie RED diverse a code con priorità diverse. I pacchetti a priorità più bassa vengono scartati prima, proteggendo il traffico più critico durante la congestione.

33.3 Traffic Shaping

Il **Traffic Shaping** è una tecnica che regola il flusso di traffico *prima* che entri nella rete, per garantire che rispetti i parametri concordati.

- **Parametri controllati:**
 - Tasso medio di pacchetti.
 - Picco massimo di traffico.
 - Dimensione massima del burst (raffica di pacchetti).

33.3.1 Token Bucket

È il meccanismo più comune per implementare il traffic shaping.

1. Un "secchio" (*bucket*) viene riempito con dei "gettoni" (*token*) a un tasso costante.
2. Per trasmettere un pacchetto, è necessario "spendere" un token.
3. Se il bucket è vuoto, il pacchetto deve attendere che venga generato un nuovo token.

Questo meccanismo impone un limite al tasso medio di trasmissione e permette di gestire i burst di traffico (fino alla capienza massima del bucket, quindi non gestisce il traffico dei singoli flussi ma il traffico totale).

33.3.2 Call Admission Control (CAC)

È una politica che decide se accettare o rifiutare un nuovo flusso di traffico in base alle risorse di rete disponibili, per garantire che la QoS dei flussi esistenti non venga compromessa. garantisce che solo il traffico conforme ai parametri contrattuali venga accettato.

33.4 struttura protocolli router

1. CAC iniziale per filtrare le connessioni e negarle se non è possibile supportarle
2. traffic shaping (token bucket)
3. WRED (controllo aggiuntivo o utile se un flusso per qualche motivo non è soggetto al traffic shaping)

34 IPv6: Caratteristiche e Transizione da IPv4

IPv6 è l'evoluzione di IPv4, progettato per risolvere il problema dell'esaurimento degli indirizzi e per migliorare l'efficienza e la sicurezza del protocollo.

34.1 Caratteristiche Principali e Differenze con IPv4

- **Indirizzi a 128 bit:** Risolve il problema della scarsità di indirizzi (da 2^{32} a 2^{128}).
- **Header Semplificato:** L'header IPv6 è più snello e ha un numero fisso di campi. I campi opzionali sono gestiti tramite *Header di Estensione*.
- **Traffic Class e Flow Label:** Campi per supportare la QoS e per identificare flussi di pacchetti che richiedono lo stesso tipo di gestione.
- **Next Header:** Campo che indica quale header segue quello IPv6 (es. TCP, UDP, o un header di estensione), creando una catena flessibile di header.
- **Nessuna frammentazione dai router:** La frammentazione dei pacchetti, se necessaria, viene gestita solo dall'host sorgente, riducendo il carico di lavoro sui router intermedi.

34.1.1 Formato e Struttura Gerarchica degli Indirizzi

Gli indirizzi IPv6 sono progettati per un routing gerarchico ed efficiente, riducendo la dimensione delle tabelle di routing globali. La struttura è spesso suddivisa in:

- **Prefisso di routing globale:** Assegnato a un ISP o a una grande organizzazione.
- **ID di sottorete:** Utilizzato per definire le sottoreti all'interno dell'organizzazione.
- **ID di interfaccia:** Identifica un dispositivo specifico all'interno della sottorete.

34.2 Meccanismi di Compatibilità e Transizione

La transizione da IPv4 a IPv6 è un processo graduale che richiede meccanismi di coesistenza.

34.2.1 Dual Stack

Un dispositivo (host o router) implementa entrambi gli stack protocollari, IPv4 e IPv6. Può quindi comunicare nativamente con host IPv4 usando IPv4 e con host IPv6 usando IPv6. È la soluzione più completa ma richiede il supporto di entrambi i protocolli su tutta l'infrastruttura.

34.2.2 Tunneling

Permette a due host IPv6 di comunicare attraverso una rete IPv4.

1. Un router di confine riceve un pacchetto IPv6.
2. **Incapsula** il pacchetto IPv6 all'interno di un pacchetto IPv4.
3. Il pacchetto IPv4 attraversa la rete intermedia.
4. Il router di confine di destinazione **decapsula** il pacchetto, estraendo quello IPv6 originale.

34.2.3 Traduzione (NAT64)

Un gateway traduce gli header dei pacchetti da IPv6 a IPv4 e viceversa, permettendo a un host solo-IPv6 di comunicare con un host solo-IPv4. Questa soluzione è più complessa e può introdurre problemi di compatibilità con alcuni protocolli.

35 Protocollo TCP (Transmission Control Protocol)

Il **Transmission Control Protocol (TCP)** è un protocollo di livello trasporto orientato alla connessione, che fornisce un servizio di trasferimento dati **affidabile** e **ordinato** tra processi applicativi. La sua architettura è progettata per gestire flussi di comunicazione separati, correggere errori e controllare il flusso dei dati.

35.1 Gestione delle Connessioni Multiple su un Server

Un server (es. web server sulla porta 80) deve poter gestire più client contemporaneamente. Per farlo, adotta un meccanismo basato su processi figli e porte dinamiche:

1. **Accettazione della connessione e `fork()`:** Quando un client si connette alla porta well-known (es. 80), il server accetta la connessione e crea un **processo figlio** dedicato tramite la chiamata di sistema `'fork()'`.
2. **Assegnazione di una porta dinamica:** Il processo figlio non continua a usare la porta 80. Il sistema operativo gli assegna una nuova **porta effimera** (superiore a 1024) per la comunicazione con quel client specifico.
3. **Redirezione del traffico:** La porta 80 del server torna immediatamente libera per accettare nuove connessioni. Il sistema operativo mantiene una tabella che mappa la connessione del client (identificata da IP e porta sorgente) alla nuova porta dinamica del processo figlio.

In questo modo, il server può gestire centinaia di connessioni simultanee senza conflitti.

35.2 Processo di Connessione: Primitiva Socket

La comunicazione TCP si basa sull'uso delle **socket**, che rappresentano gli endpoint della comunicazione.

35.2.1 Lato Client

1. **Risoluzione DNS:** Il client ottiene l'indirizzo IP del server tramite una richiesta DNS.
2. **Creazione della Socket:** Utilizza la primitiva `'socket()'` per ottenere un descrittore di file per la comunicazione.
3. **Connessione:** Invoca `'connect()'`, specificando l'IP e la porta del server. Questa chiamata avvia il *three-way handshake* e il processo client si blocca in attesa che la connessione venga stabilita.

35.2.2 Lato Server

1. **Creazione della Socket:** Il server crea una socket con `'socket()'`.
2. **Binding:** Usa `'bind()'` per associare la socket a un indirizzo IP e a una porta specifica (es. 80).
3. **Ascolto:** Con `'listen()'`, il server si mette in ascolto e definisce la dimensione della coda per le connessioni in attesa.
4. **Accettazione:** La chiamata `'accept()'` blocca il server finché non arriva una richiesta di connessione da un client. Quando arriva, `'accept()'` restituisce una nuova socket dedicata a quella specifica connessione.

35.2.3 Scambio Dati

Una volta stabilita la connessione, entrambi gli endpoint usano le primitive `'send()'` e `'receive()'` per scambiare dati in modo bidirezionale.

35.3 Header TCP

L'header TCP ha una dimensione minima di 20 byte e contiene campi cruciali per garantire una comunicazione affidabile.

- **Source Port e Destination Port** (16 bit ciascuno): Identificano i processi applicativi del mittente e del destinatario.
- **Sequence Number** (32 bit): Essendo TCP un protocollo *byte-stream*, questo campo indica il numero di sequenza del **primo byte** di dati nel segmento.
- **Acknowledgment Number** (32 bit): Indica il numero di sequenza del **prossimo byte** che il destinatario si aspetta di ricevere. Funge da conferma di ricezione.

- **Control Flags** (es. SYN, ACK, FIN, RST): Bit di controllo per gestire le fasi della connessione (apertura, chiusura, reset).
- **Window Size** (16 bit): Campo fondamentale per il **controllo di flusso**. Indica la quantità di spazio (in byte) disponibile nel buffer di ricezione del mittente di questo segmento.
- **Checksum** (16 bit): Usato per la verifica dell'integrità dell'header e dei dati.

35.3.1 Pseudo-Header e Checksum

Per garantire che un segmento TCP sia arrivato alla destinazione corretta (e non sia stato, ad esempio, reindirizzato erroneamente), la checksum viene calcolata su una struttura virtuale chiamata **pseudo-header**, che include:

- Indirizzo IP sorgente e destinazione.
- Codice del protocollo (6 per TCP).
- Lunghezza del segmento TCP.

Questo meccanismo lega il segmento TCP al pacchetto IP che lo trasporta, aumentandone l'affidabilità.

35.4 Fase di Apertura: Three-Way Handshake

La connessione TCP viene stabilita tramite una procedura a tre passaggi:

1. **Client → Server (SYN)**: Il client invia un segmento con il flag **SYN** (Synchronize) attivo e un numero di sequenza iniziale casuale (es. 'SEQ=x').
2. **Server → Client (SYN-ACK)**: Il server risponde con un segmento che ha sia il flag **SYN** che **ACK** attivi.
 - Il suo numero di sequenza è un nuovo valore casuale (es. 'SEQ=y').
 - Il numero di acknowledgment è 'ACK=x+1', per confermare la ricezione del primo segmento del client.
3. **Client → Server (ACK)**: Il client invia un ultimo segmento con il flag **ACK** attivo per confermare la ricezione del SYN del server. Il numero di acknowledgment è 'ACK=y+1'.

A questo punto la connessione è stabilita (*ESTABLISHED*) e il trasferimento dati può iniziare.

35.5 Fase di Trasferimento Dati e Controllo di Flusso

Il controllo di flusso previene che un mittente veloce sovraccarichi un ricevitore lento.

- **Sliding Window**: Il mittente può inviare una quantità di dati pari alla **Window Size** comunicata dal ricevitore.
- **Aggiornamento Dinamico**: Man mano che il ricevitore elabora i dati dal suo buffer, invia ACK con un valore di 'Window Size' aggiornato, comunicando lo spazio libero.
- **Zero-Window**: Se il ricevitore comunica una 'Window Size = 0', il mittente deve fermarsi.

35.5.1 Gestione dei Blocchi: Persist Timer

Se un ACK con un aggiornamento della 'Window Size' (da 0 a un valore positivo) viene perso, il mittente potrebbe rimanere bloccato indefinitamente. Per evitare ciò:

- Quando il mittente riceve 'Window Size = 0', avvia un **Persist Timer**.
- Alla scadenza del timer, invia un piccolo pacchetto **probe** per chiedere un aggiornamento dello stato della finestra. Questo garantisce che la comunicazione non si blocchi.

35.6 Gestione delle Connessioni Inattive: Keep-Alive Timer

Per rilevare se una connessione è ancora attiva anche in assenza di traffico, si usa il **Keep-Alive Timer**.

- Se una connessione rimane inattiva per un lungo periodo, il server inizia a inviare periodicamente dei pacchetti **probe** (sondaggi).
- Se non riceve risposta dopo un certo numero di tentativi, il server assume che il client si sia disconnesso in modo anomalo e chiude la connessione, liberando le risorse.
- La configurazione di questo timer è un compromesso: un valore troppo basso può chiudere connessioni valide ma lente, mentre uno troppo alto spreca risorse del server.

36 Controllo di Congestione in TCP

36.1 Meccanismi di Ritrasmissione: RTO vs Fast Retransmit

Entrambi i meccanismi rilevano la perdita di pacchetti, ma reagiscono in modi diversi.

- **Retransmission Timeout (RTO):**
 - **Trigger:** Scadenza di un timer di attesa per un ACK.
 - **Significato:** Segnale di un problema potenzialmente grave (segmento perso, ACK perso o forte ritardo).
 - **Azione:** Il mittente ritrasmette il segmento non confermato. È un meccanismo di sicurezza fondamentale ma lento.
- **Fast Retransmit:**
 - **Trigger:** Ricezione di **tre ACK duplicati** per lo stesso numero di sequenza.
 - **Significato:** Segnale forte che un singolo segmento è stato perso, ma la rete è ancora funzionante (i pacchetti successivi stanno arrivando).
 - **Azione:** Il mittente ritrasmette il segmento perso *immediatamente*, senza attendere la scadenza dell'RTO.

36.2 Ottimizzazione del Flusso: Algoritmi di Nagle e Clark

Questi algoritmi **auto-adattivi** riducono l'overhead in scenari di comunicazione asimmetrica.

infatti quando un pacchetto viene inviato dal mittente (ad esempio un carattere da scrivere su un terminale remoto), il ricevente invia altri 3 messaggi:

- ACK (Acknowledgment) per confermare la ricezione del carattere
- Window Size(WS) per aggiornare la finestra di ricezione
- Echo del carattere per visualizzarlo sul terminale remoto

36.2.1 per evitare di intasare la banda per ogni singolo carattere si usano questi 2 metodi:

- **Algoritmo di Nagle (lato Mittente):**
 - **Problema:** Mittente lento che invia molti piccoli pacchetti (alto overhead).
COME:
 - * se non ci sono pacchetti inviati in transito spedisce subito il pacchetto
 - * se ci sono già raggruppa tutti i pacchetti da inviare in uno singolo e verrà inviato non appena si riceverà l'ACK del precedente
- **Algoritmo di Clark (lato Ricevitore):**
 - **Problema:** Ricevitore lento che libera il buffer di pochi byte alla volta, causando l'invio di piccoli aggiornamenti di finestra e inducendo il mittente a inviare piccoli pacchetti.
COME:
 - * invece di inviare un ACK (con anche il WS) ogni volta che si libera uno spazio (si parte da una situazione di buffer pieno) aspetta un intervallo di tempo
- **SELF-CLOCKING:** è il metodo implementato automaticamente in TCP che usa sia nagle che clark

36.3 Controllo della Congestione (Congestion Window - cwnd)

TCP adatta la sua velocità di trasmissione non solo al ricevitore ma anche allo stato della rete, usando una finestra di congestione ('cwnd'). Il suo valore viene regolato dinamicamente.

36.3.1 Fasi di Crescita della Finestra

- **Slow Start:** All'inizio della connessione. La 'cwnd' parte da 1 MSS (un pacchetto alla volta) e cresce **esponenzialmente** (raddoppia a ogni RTT). L'obiettivo è trovare rapidamente la capacità della rete. Questa fase dura finché 'cwnd' non raggiunge la soglia 'sssthresh'.
- **Congestion Avoidance:** Superata la soglia 'sssthresh'. La crescita della 'cwnd' diventa **lineare** (aumenta di 1 MSS per RTT), sondando la rete con più cautela per evitare di creare congestione.

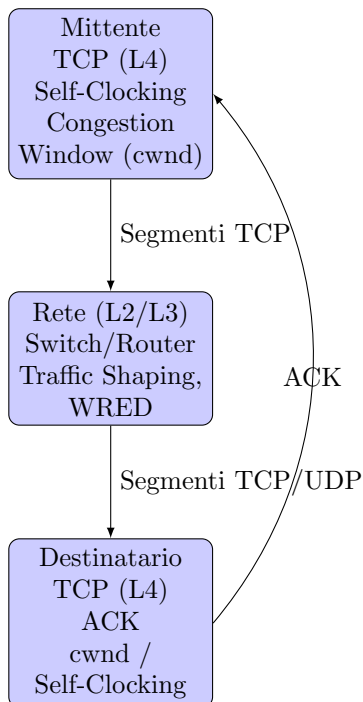
36.3.2 Reazione alla Perdita di Pacchetti (Congestione)

La perdita è il principale segnale di congestione. La reazione di TCP è diversa a seconda della gravità del segnale.

- **Reazione a Timeout (RTO):** Segnale di **congestione grave**.
 - ‘ssthresh’ viene dimezzato.
 - ‘cwnd’ viene resettata a **1 MSS**.
 - La trasmissione riparte dalla fase di **Slow Start**.
- **Reazione a Fast Retransmit (3 ACK duplicati):** Segnale di **congestione lieve**.
 - ‘ssthresh’ viene dimezzato.
 - ‘cwnd’ viene impostata al nuovo valore di ‘ssthresh’ (invece di 1).
 - La trasmissione riprende direttamente dalla fase di **Congestion Avoidance** (meccanismo di **Fast Recovery**).

36.3.3 self-clocking VS cwnd

- SELF-CLOCKING si occupa di evitare l’intasamento di mittente e ricevente
- CWND si occupa di evitare l’intasamento della rete



37 Meccanismi Avanzati in TCP

Per ottimizzare le performance e l'affidabilità, TCP utilizza algoritmi sofisticati per la gestione dei timer, delle ritrasmissioni e della chiusura delle connessioni.

37.1 Calcolo Dinamico del Timeout (RTO)

L'efficienza di TCP dipende da un valore di **Retransmission Timeout (RTO)** ben calibrato. TCP lo calcola dinamicamente per adattarsi alle condizioni della rete.

- **Obiettivo:** Evitare ritrasmissioni premature (RTO troppo basso) o attese inutili (RTO troppo alto).
- **Metodo:** Si basa su una stima del Round Trip Time (RTT) e della sua variazione (deviazione), calcolate tramite medie pesate per smorzare le fluttuazioni istantanee.
 - **Smoothed RTT:** $RTT_{nuovo} = \alpha \cdot RTT_{vecchio} + (1 - \alpha) \cdot M$ (nuova misura dell'RTT)
 - **Smoothed Deviation:** $D_{nuova} = \alpha \cdot D_{vecchia} + (1 - \alpha) \cdot |M - RTT_{nuovo}|$
 - **Calcolo finale RTO:** $RTO = RTT_{nuovo} + 4 \cdot D_{nuova}$ (4 per dare spazio a eventuali ritardi)
- **Casi Iniziali:**
 - **Primo segmento:** RTO è un valore di default (es. 3 secondi).
 - **Secondo segmento:** $RTO = M + 4 \times (\frac{M}{2})$ (poichè non ho D zero e RTT zero)
 - **Dal terzo in poi:** Si usano le formule complete.

37.2 Regola di Karn: Gestire l'Ambiguità nelle Ritrasmissioni

- **Problema:** Quando un segmento viene ritrasmesso, l'ACK che arriva è ambiguo: si riferisce alla prima o alla seconda trasmissione? Usare questo RTT misurato può portare a stime di RTO completamente errate.
- **Soluzione (Regola di Karn):**
 1. **Ignorare la misura:** Se un segmento è stato ritrasmesso, la misura di RTT per quell'ACK viene ignorata.
 2. **Applicare l'Exponential Backoff:** Invece di ricalcolare l'RTO, lo si raddoppia: $RTO_{nuovo} = 2 \cdot RTO_{vecchio}$. Questo approccio conservativo aiuta a gestire la congestione.
(serve solo a ignorare la misurazione se il pacchetto viene ritrasmesso non per misurarlo correttamente, e per garantire il tempo necessario per il pacchetto)

37.3 Migliorare l'Affidabilità: Timestamp e SACK

- **Opzione TCP Timestamp:** Per superare l'ambiguità di Karn, l'opzione Timestamp permette al mittente di inserire un timestamp nel segmento. Il ricevitore inserisce il Timestamp di invio nell'ACK corrispondente, il mittente poi quando riceverà l'ACK lo associerà attraverso i Timestamp dei pacchetti al suo corrispondente e calcolerà il tempo totale attraverso il Timestamp di arrivo dell'ACK.
- **Selective Acknowledgment (SACK):**
 - **Problema dell'ACK Cumulativo:** Se viene perso il pacchetto 5 ma arrivano i pacchetti 6, 7 e 8, l'ACK cumulativo continuerà a richiedere il 5, senza informare il mittente che i pacchetti successivi sono già arrivati.
 - **Soluzione SACK:** Il ricevitore, oltre all'ACK cumulativo, può specificare blocchi di dati non contigui ricevuti correttamente (es. "Ho ricevuto i blocchi 6-8"). Questo permette al mittente di ritrasmettere **solo i segmenti mancanti**, migliorando drasticamente l'efficienza in caso di perdite multiple.

37.4 Chiusura della Connessione (Four-Way Handshake)

La chiusura di una connessione TCP è un processo a quattro fasi per garantire che entrambi i lati abbiano terminato la trasmissione.

1. **Host A → Host B (FIN):** L'host che inizia la chiusura (chiusura attiva) invia un segmento con flag FIN.
2. **Host B → Host A (ACK):** L'host B conferma la ricezione del FIN. Da questo momento, l'host A non può più inviare dati, ma B sì (*half-close*).
3. **Host B → Host A (FIN):** Quando anche B ha finito di trasmettere i suoi dati, invia il suo segmento FIN.

4. **Host A** $\rightarrow$ **Host B** (**ACK**): L'host A conferma il FIN di B e entra nello stato **TIME-WAIT**.

Stato TIME-WAIT: Stato cruciale in cui rimane l'host che ha iniziato la chiusura. Dura un tempo definito (es. 2 minuti) per assicurare che eventuali pacchetti duplicati della vecchia connessione vengano scartati e non interferiscano con una nuova connessione.

38 UDP (User Datagram Protocol)

UDP è un protocollo di trasporto alternativo a TCP, caratterizzato dalla sua semplicità e velocità a scapito dell'affidabilità.

38.1 Caratteristiche Principali

- **Senza connessione (Connectionless):** Non c'è handshake di apertura o chiusura. I datagrammi vengono inviati direttamente.
- **Best-effort:** Non offre alcuna garanzia su consegna, ordine o assenza di duplicati. Non implementa controllo di flusso né di congestione.
- **Overhead Minimo:** L'header è di soli 8 byte, rendendolo estremamente leggero ed efficiente.
- **Checksum Opzionale:** Utilizza uno pseudo-header (come TCP) per includere gli indirizzi IP nel calcolo della checksum, ma il suo utilizzo è opzionale.

38.2 Quando Usare UDP?

UDP è la scelta ideale per applicazioni dove la **velocità** e la **bassa latenza** sono più importanti della completa affidabilità. La gestione di eventuali perdite è delegata all'applicazione.

- **Streaming audio/video** (es. VoIP, videoconferenze): Perdere un singolo pacchetto è preferibile a introdurre ritardi per la ritrasmissione.
- **Gaming online:** La latenza minima è cruciale.
- **Servizi di query veloci** (es. DNS): La richiesta e la risposta sono piccole e devono essere rapide.

39 Protocolli Real-Time: RTP e RTCP

Per la trasmissione di dati multimediali (audio/video), che richiedono bassa latenza più che assoluta affidabilità, si utilizza una coppia di protocolli specifici: RTP per i dati e RTCP per il controllo.

39.1 Caratteristiche Generali

- **Separazione Dati/Controllo:** RTP (Real-time Transport Protocol) trasporta i flussi multimediali, mentre RTCP (RTP Control Protocol) gestisce il monitoraggio e le statistiche della sessione.
- **Basato su UDP:** Entrambi i protocolli operano su UDP per minimizzare la latenza, sacrificando l'affidabilità intrinseca di TCP (niente ritrasmissioni automatiche).
- **Funzionamento a Coppie:** Ogni flusso RTP è sempre associato a un flusso RTCP sulla porta successiva.

39.2 RTP (Real-time Transport Protocol) - Il Flusso Dati

Il suo compito è incapsulare i dati multimediali e fornire le informazioni necessarie per la loro corretta riproduzione.

- **Header RTP - Campi Chiave:**
 - **Sequence Number:** Numera i pacchetti per permettere al ricevitore di riordinarli e di rilevare eventuali perdite.
 - **Timestamp:** Indica l'istante di campionamento del primo byte del pacchetto. È fondamentale per la sincronizzazione e per calcolare il *jitter* (variazione del ritardo).
 - **Payload Type (PT):** Identifica il tipo di codifica utilizzata (es. G.711 per l'audio, H.264 per il video).
 - **Synchronization Source (SSRC):** Un identificatore numerico univoco per ogni sorgente di flusso all'interno di una sessione.

39.3 RTCP (RTP Control Protocol) - Il Flusso di Controllo

RTCP opera in parallelo a RTP per fornire feedback sulla qualità della trasmissione.

- **Funzioni Principali:**
 - **Monitoraggio della QoS:** I ricevitori inviano periodicamente report al mittente con statistiche su pacchetti persi, jitter e Round-Trip Time.
 - **Sincronizzazione:** Fornisce i dati (tramite timestamp NTP) per sincronizzare flussi diversi (es. audio e video della stessa sorgente).
 - **Feedback e Adattamento:** Basandosi sui report RTCP, un mittente intelligente può adattare la qualità del flusso (es. cambiando codec o bitrate) in base alle condizioni della rete.

40 DNS (Domain Name System)

Il DNS è il sistema distribuito che traduce i nomi di dominio leggibili dall'uomo (es. `www.google.com`) in indirizzi IP numerici (es. `142.250.184.68`), necessari per l'instradamento dei pacchetti in rete. Opera a livello applicativo (Livello 7).

40.1 Struttura Gerarchica e Risoluzione dei Nomi

- **Struttura ad Albero:** Il DNS ha una struttura gerarchica che parte da una radice (*root*) e si dirama in Top-Level Domains (TLD, es. `.com`, `.it`), domini di secondo livello (es. `google`, `unimi`) e sottodomini.
- **Server DNS Autoritativi:** Per ogni dominio esiste almeno un server autoritativo che detiene i record ufficiali per quel dominio. (ad esempio `.di` `.edu` `.com` `.org` ecc.)
- **Risoluzione Ricorsiva vs Iterativa:**

- **Ricorsiva:** Il client chiede al suo DNS resolver locale di trovare l'IP. Il resolver fa tutto il lavoro, contattando gli altri server DNS necessari, e restituisce al client la risposta finale.

ES: il mio pc chiede al suo server DNS di riferimento, il quale (se non ha la risposta) chiede al DNS root il quale chiede al TDL e così via, ricevuta la risposta si fa il processo opposto e si risponde al DNS precedente fino al mio pc.

- **Iterativa:** Il resolver locale chiede al server root, che lo reindirizza al server TLD (Un TLD (Top-Level Domain) è la parte finale di un nome di dominio, quella che segue l'ultimo punto. Pensa a `.com`, `.it`, `.org`). Il resolver chiede al server TLD, che lo reindirizza al server autoritativo, e così via, fino a ottenere la risposta.

ES: mio pc chiede al suo DNS di riferimento il quale gli dà l'ip del DNS root, allora il mio pc contatta il DNS root che gli dà l'ip di un TDL e così via.

- **Caching:** Per velocizzare le richieste, i server DNS memorizzano temporaneamente (in una **cache**) le risposte ottenute. La durata della permanenza in cache è definita dal campo **TTL** (time to live) del record.

40.2 Record DNS

Il database del DNS è composto da una tabella di cache che memorizza per un tempo determinato le vecchie richieste (per evitare di fare sempre le stesse domande) popolate da **Record di Risorse (RR)**, che associano un nome a un valore. I tipi più comuni sono:

- **A:** Associa un nome a un indirizzo **IPv4**.
- **AAAA** ("quad-A"): Associa un nome a un indirizzo **IPv6**.
- **MX** (Mail Exchange): Specifica qual è il mail server responsabile per un dominio.
- **CNAME** (Canonical Name): Crea un alias, facendo puntare un nome a un altro nome.
- **NS** (Name Server): Indica quale server DNS è autoritativo per un dominio.
- **TXT:** Permette di inserire testo arbitrario, usato spesso per meccanismi di sicurezza (es. SPF).

41 Architettura della Posta Elettronica (Email)

Il servizio di posta elettronica è un sistema distribuito per lo scambio di messaggi, basato su componenti e protocolli standardizzati.

41.1 Componenti Chiave

- **User Agent (UA)**: Il client di posta (es. Outlook, Gmail) usato per comporre e leggere le email.
- **Message Transfer Agent (MTA)**: Il server di posta che si occupa di instradare e trasferire i messaggi tra domini diversi.
- **Message Store**: Lo spazio su server dove le email vengono archiviate in attesa di essere lette dal destinatario.

41.2 Flusso di Invio e Ricezione di un'Email

1. **Invio dal Client**: L'utente scrive un'email con il suo UA e la invia al proprio MTA server (spesso via SMTP).
2. **Trasferimento Server-to-Server (SMTP)**: L'MTA del mittente contatta l'MTA del destinatario sulla porta 25 e trasferisce il messaggio utilizzando il protocollo **SMTP (Simple Mail Transfer Protocol)**.
3. **Archiviazione**: L'MTA del destinatario riceve l'email e la deposita nel Message Store dell'utente.
4. **Lettura da parte del Destinatario**: L'utente si connette al suo server di posta tramite il suo UA per leggere l'email.

41.3 Protocolli per l'Accesso alla Posta

Per la fase di lettura, l'utente usa uno di questi due protocolli:

- **POP3 (Post Office Protocol v3)**: Scarica le email dal server al dispositivo locale, tipicamente **rimuovendole** dal server. Adatto per l'accesso da un singolo dispositivo.
- **IMAP (Internet Message Access Protocol)**: Gestisce le email **direttamente sul server**. Le modifiche (lettura, cancellazione, spostamento in cartelle) sono sincronizzate su tutti i dispositivi. È lo standard per l'accesso multi-dispositivo.

41.4 Gestione degli Allegati: MIME e Base64

- **Il Problema**: SMTP è un protocollo testuale che usa una codifica ASCII a 7 bit per i caratteri, con byte speciali per i comandi di controllo (es. invio a capo, spazio, ecc.), chiamata **NVT (Network Virtual Terminal)**. SMTP/NVT non può trasportare direttamente dati binari come immagini, video o PDF.
- **La Soluzione - MIME (Multipurpose Internet Mail Extensions)**: È un'estensione di SMTP che aggiunge header per descrivere il contenuto non testuale senza modificare il protocollo.
 - **Content-Type**: Specifica il tipo di file (es. `image/jpeg`).
 - **Content-Transfer-Encoding**: Indica come i dati binari sono stati codificati per essere trasmessi come testo.
- **La Codifica - Base64**: È l'algoritmo più comune usato da MIME. Converte i dati binari in una sequenza di caratteri ASCII stampabili, rendendoli sicuri per la trasmissione via SMTP.
 - I dati binari vengono presi in blocchi da 3 byte (24 bit) e divisi in 4 blocchi da 6 bit ciascuno.
 - Ogni blocco da 6 bit viene convertito in un carattere ASCII utilizzando la tabella Base64 (64 possibili caratteri).
 - Se i dati non sono multipli di 3 byte, si usa il padding con il carattere `=` per completare il gruppo.

41.5 struttura SMTP

- tutto scritto in formato NVT
- header + messaggio (solo testuale)
- le immagini vengono aggiunte all'header come estensione

42 FTP (File Transfer Protocol)

FTP è un protocollo standard per il trasferimento di file tra computer su una rete, basato su TCP per garantire l'affidabilità.

42.1 Architettura a Due Connessioni

La caratteristica distintiva di FTP è l'uso di due canali di comunicazione separati:

- **Connessione di Controllo (porta 21 del server):**
 - Canale persistente, rimane aperto per tutta la sessione.
 - Utilizzato per inviare comandi (es. 'USER', 'PASS', 'LIST', 'GET') e ricevere risposte dal server.
- **Connessione Dati (porta 20 del server ATTENZIONE: solo in connessione FTP attiva):**
 - Canale temporaneo, aperto solo per il trasferimento effettivo di un file o di una lista di directory.
 - Viene chiusa al termine di ogni trasferimento.

42.2 Modalità di Connessione Dati: Attiva vs Passiva

(fanno la stessa cosa ma la passiva è la più usata dato che risolve il problema dei firewall)

- **FTP Attivo:**
 1. Il client si connette alla porta 21 del server (controllo).
 2. Quando deve trasferire dati, il client comunica al server una sua porta su cui mettersi in ascolto.
 3. Il **server inizia la connessione dati** dalla sua porta 20 verso la porta specificata dal client.
 4. *Problema:* Spesso bloccato dai firewall lato client.
- **FTP Passivo:**
 1. Il client si connette alla porta 21 del server (controllo).
 2. Quando deve trasferire dati, il client invia il comando 'PASV'.
 3. Il server risponde indicando una sua porta (non la 20) su cui si mette in ascolto.
 4. Il **client inizia la connessione dati** verso la porta specificata dal server.
 5. *Vantaggio:* Più compatibile con i firewall.

42.3 Separazione tra Piano di Controllo e Piano Dati

L'architettura di FTP è un classico esempio di separazione dei piani:

- **Piano di Controllo:** La connessione sulla porta 21, gestisce la logica della sessione (autenticazione, comandi).
- **Piano Dati:** La connessione temporanea, gestisce unicamente il trasporto dei bit del file.

43 Fasi di Assegnazione IP in DHCP

Il protocollo **DHCP** (Dynamic Host Configuration Protocol) permette ai client di ottenere automaticamente un indirizzo IP e altri parametri di rete. L'assegnazione avviene in quattro fasi principali:

1. DHCP Discover:

- Il client, non conoscendo un server DHCP, invia un messaggio broadcast sulla rete locale per scoprire server disponibili.
- Questo messaggio contiene l'indirizzo MAC del client e altre informazioni opzionali.

2. DHCP Offer:

- I server DHCP ricevono la richiesta Discover e rispondono con un messaggio *Offer*, proponendo un indirizzo IP disponibile e i parametri di configurazione (subnet mask, gateway, DNS, lease time).
- L'Offer è inviato come broadcast o unicast al client.

3. DHCP Request:

- Il client seleziona un'offerta tra quelle ricevute e invia un messaggio *Request* a tutti i server DHCP, indicando quale offerta accetta.
- Questo serve anche a informare gli altri server di non assegnare quell'indirizzo IP ad altri client.

4. DHCP Acknowledge (ACK):

- Il server DHCP che ha ricevuto il Request conferma l'assegnazione con un messaggio *ACK*.
- Il client può ora configurare la propria interfaccia di rete con l'indirizzo IP e gli altri parametri forniti.
- Il lease ha una durata definita, al termine della quale il client deve rinnovarlo.

44 HTTP (HyperText Transfer Protocol)

HTTP è il protocollo a livello applicativo su cui si fonda il World Wide Web. Opera con un modello **richiesta-risposta** (request-reply): un client richiede una risorsa (es. una pagina web) e un server risponde fornendola. La sua evoluzione si è concentrata sulla risoluzione dei colli di bottiglia per migliorare la velocità e l'efficienza.

44.1 HTTP/1.0: Connessioni Non Permanenti

La prima versione di HTTP utilizzava un modello semplice ma inefficiente.

- **Funzionamento:** Per ogni singolo oggetto richiesto (es. un'immagine, un file CSS) veniva aperta una nuova connessione TCP, che veniva chiusa subito dopo la risposta.
- **Problemi Principali:**
 - **Overhead Elevato:** L'apertura e chiusura continua di connessioni TCP (con il relativo three-way handshake) consumava molte risorse del server e introduceva latenza.
 - **Inefficienza di TCP:** Le connessioni, essendo molto brevi, non beneficiavano mai appieno dei meccanismi di controllo della congestione di TCP (operavano sempre in fase di *slow start*).

44.2 HTTP/1.1: Connessioni Permanenti e Pipelining

HTTP/1.1 ha introdotto due migliorie fondamentali per risolvere i problemi della versione 1.0.

- **Connessioni Permanenti (Keep-Alive):** La connessione TCP viene mantenuta aperta per più richieste consecutive, ammortizzando i costi dell'handshake e migliorando le performance.
- **Pipelining:** Il client può inviare più richieste in sequenza sulla stessa connessione senza attendere la risposta della precedente.
- **Nuovo Problema - Head-of-Line (HoL) Blocking (Livello 7):** Se la prima richiesta nella pipeline è lenta da processare, tutte le risposte successive vengono bloccate, anche se sono già pronte.

44.3 HTTP/2: Multiplexing su TCP

HTTP/2 ha rivoluzionato la gestione delle richieste per risolvere l'HoL blocking di HTTP/1.1.

- **Multiplexing:** Più flussi di richiesta-risposta (*stream*) vengono trasmessi in parallelo sulla **stessa singola connessione TCP**. Una risposta lenta in uno stream non blocca gli altri.
- **Altre Migliorie:**
 - **Server Push:** Il server può inviare proattivamente risorse che sa che il client richiederà a breve (es. CSS e JS di una pagina HTML).
 - **Compressione degli Header (HPACK):** Riduce la ridondanza e la dimensione degli header HTTP.
- **Nuovo Problema - Head-of-Line Blocking (Livello 4, TCP):** Sebbene gli stream a livello applicativo siano indipendenti, sono tutti incapsulati in un unico flusso TCP ordinato. La perdita di un singolo pacchetto TCP blocca la consegna di **tutti gli stream** finché quel pacchetto non viene ritrasmesso.

44.4 HTTP/3: Il Passaggio a QUIC su UDP

Per superare il limite intrinseco di TCP, HTTP/3 cambia radicalmente il protocollo di trasporto sottostante.

- **Basato su QUIC (Quick UDP Internet Connections):** HTTP/3 non usa TCP, ma QUIC, un nuovo protocollo di trasporto che opera su UDP.
 - **Vantaggi di QUIC:**
 - **Risolve l'HoL Blocking di TCP:** QUIC gestisce gli stream in modo nativo e indipendente. La perdita di un pacchetto relativo a uno stream non blocca gli altri.
 - **Handshake più Veloce (0-RTT):** Riduce drasticamente i tempi per stabilire o riprendere una connessione sicura.
 - **Sicurezza Integrata:** La crittografia (TLS 1.3) è integrata nativamente nel protocollo, non è uno strato aggiuntivo.
- con QUIC si introducono più flussi udp con un modello TCP costruito dentro.

Protocollo	Caratteristica Chiave	Problema Risolto	Nuovo Limite Introdotto
HTTP/1.0	Connessioni non permanenti	-	Alto overhead, inefficienza
HTTP/1.1	Connessioni permanenti	Overhead di connessione	HoL Blocking (Applicativo)
HTTP/2	Multiplexing su TCP	HoL Blocking (Applicativo)	HoL Blocking (Trasporto/TCP)
HTTP/3	Multiplexing su QUIC/UDP	HoL Blocking (Trasporto/TCP)	-

Tabella 1: Evoluzione delle performance del protocollo HTTP.

45 Differenza tra HTTP e HTTPS

La principale differenza tra **HTTP** e **HTTPS** risiede nella sicurezza e nella cifratura dei dati.

- **HTTP** (*Hypertext Transfer Protocol*):
 - Non sicuro: i dati vengono trasmessi in chiaro e sono vulnerabili all'intercettazione.
 - Utilizza la porta **80**.
- **HTTPS** (*Hypertext Transfer Protocol Secure*):
 - Sicuro: utilizza la crittografia **SSL/TLS** per proteggere i dati.
 - Utilizza la porta **443**.
 - Richiede un certificato di sicurezza per stabilire una connessione cifrata.

In sintesi, mentre HTTP fornisce un trasferimento dati non protetto, HTTPS garantisce la riservatezza e l'integrità dei dati tramite la cifratura.